State Estimation and Mapping

Inspired by TTIC 31170 by Dr. Matthew Walter.

Mihir Rao

Contents

1 Introduction and Graphical Models **3**
- 1.1 Robots must reason under uncertainty . . . 3
- 1.2 Probability as a language for uncertainty . . . 4
- 1.3 Why graphical models? . . . 5
- 1.4 Bayesian networks and directed separation . . . 6
- 1.5 Markov random fields and undirected separation . . . 8
- 1.6 Converting a Bayesian network to an MRF . . . 10
- 1.7 Inference with evidence: from factors to messages . . . 11
- 1.8 Variable elimination in a graph with cycles . . . 13

2 Recursive Bayesian Estimation **15**
- 2.1 Probability spaces and stochastic processes . . . 15
- 2.2 Means, covariance, and stationarity . . . 16
- 2.3 Least squares estimation . . . 18
- 2.4 The Markov assumption . . . 20
- 2.5 Hidden Markov models and forward–backward inference . . . 21
- 2.6 MAP sequences and the Viterbi algorithm . . . 24

3 Recursive Bayesian Estimation (Continued) **27**
- 3.1 Filtering, smoothing, and robot localization . . . 27
- 3.2 Learning an HMM from observations . . . 28
- 3.3 What belongs in the robot's state? . . . 31
- 3.4 Generative laws and belief distributions . . . 32
- 3.5 Deriving the Bayes filter . . . 33
- 3.6 A hallway with a door sensor . . . 34

4 Kalman Filters **36**
- 4.1 The Bayes filter, revisited . . . 36
- 4.2 Gaussian distributions . . . 37
- 4.3 The Kalman filter . . . 39
- 4.4 The extended Kalman filter . . . 42

5 Nonparametric Filters **45**
- 5.1 The discrete Bayes filter . . . 45
- 5.2 Monte Carlo approximation . . . 46
- 5.3 Sampling from distributions . . . 47
- 5.4 Importance sampling . . . 48
- 5.5 Sequential importance sampling . . . 49
- 5.6 Choosing the proposal . . . 50
- 5.7 Resampling . . . 52

6 Simultaneous Localization and Mapping **54**
6.1 What the robot knows, and what it must estimate . . . 54
6.2 Online SLAM and full SLAM . . . 56
6.3 A Gaussian representation of pose and landmarks . . . 58
6.4 Deriving the EKF prediction . . . 59
6.5 Deriving the measurement update . . . 60
6.6 Growing the map without losing shared uncertainty . . . 62
6.7 A complete one-dimensional SLAM calculation . . . 64
6.8 What limits EKF SLAM, and why pose graphs help . . . 66

7 Simultaneous Localization and Mapping (Continued) **67**
7.1 Why change the Gaussian representation? . . . 67
7.2 Deriving Gaussian information form . . . 67
7.3 What the information matrix says about a graph . . . 68
7.4 Deriving an information-form observation update . . . 70
7.5 Conditioning and marginalization are different operations . . . 71
7.6 Deriving time prediction in the information filter . . . 73
7.7 Sparsity, approximation, and uncertainty . . . 75
7.8 Keeping old poses: full SLAM as an optimization problem . . . 76
7.9 Turning nonlinear residuals into a linear least-squares step . . . 77
7.10 Worked loop closure: how a discrepancy is distributed . . . 78

Lecture 1: Introduction and Graphical Models

1.1 Robots must reason under uncertainty

Robots face uncertain state, noisy measurements, and unpredictable motion. Probability represents both current uncertainty and future outcomes.

The state contains the quantities needed to describe the system. For an underwater vehicle, these include position, orientation, linear velocity, and angular velocity. An inertial navigation system combines inertial measurements with a filter. A magnetometer helps estimate heading but can be biased; a Doppler velocity log measures velocity relative to the bottom only when sufficiently close to it. Cameras and sonar provide additional measurements.

For a mobile robot, the state may be position and heading. Wheel encoders estimate forward velocity, a laser scanner measures obstacle distance and bearing, and a camera provides images. Unmodeled dynamics and wheel slip introduce motion errors. Dead reckoning accumulates motion estimates from an initial pose, so these errors can accumulate into drift and misaligned maps.

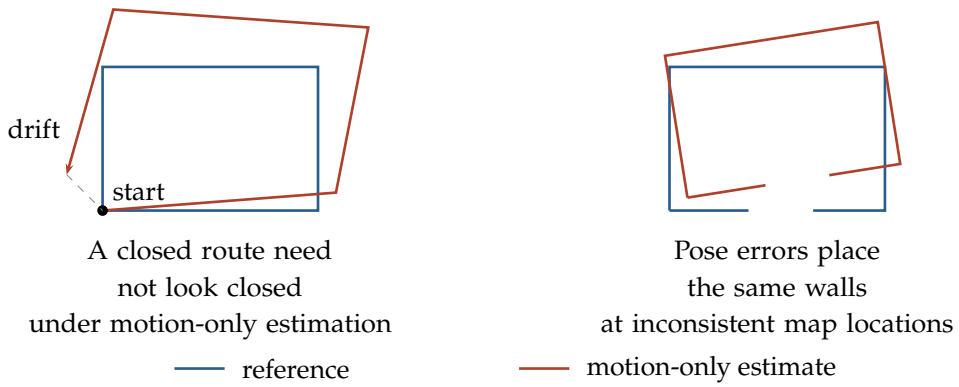


Figure 1: Two consequences of accumulated pose error. Left: motion estimates can fail to close a loop. Right: inconsistent poses can misalign repeated views of walls. The drawings are schematic.

1.2 Probability as a language for uncertainty

Definition 1.1 [Probability space]. A probability model needs three things: possible outcomes, events we can ask about, and probabilities for those events. A probability space is a triple $(\Omega, \mathcal{S}, P)$. The sample space Ω contains possible outcomes. The sigma-algebra $\mathcal{S}$ is the collection of allowed events, each a subset of Ω; it is closed under complements and countable unions. The probability measure P is the function that assigns a probability to each event.

Probabilities cannot be negative, all possible outcomes together have probability one, and disjoint events add without overlap. Formally:

$$P(A) \geq 0, \qquad P(\Omega) = 1, \qquad P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i) \quad \text{for pairwise disjoint } A_i.$$

The empty event has $P(\varnothing) = 0$. For two events, count the overlap only once:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

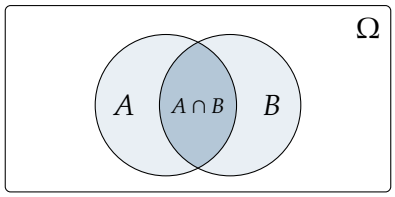


Figure 2: The intersection contributes to both $P(A)$ and $P(B)$, so inclusion–exclusion subtracts it once. Conditioning on B restricts attention to the right-hand event and renormalizes its probability to one.

Conditioning means updating our probabilities after learning that an event occurred. For $P(B) > 0$, conditional probability is

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Rearranging gives the product rule $P(A \cap B) = P(A \mid B)P(B)$. Applying this rule repeatedly yields the chain rule:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1}), \tag{1.1}$$

where the first factor is $p(x_1)$. Any ordering gives a valid chain rule; a useful graphical model allows many variables to be removed from these conditioning sets.

Theorem 1.2 [Bayes' rule]. *For events with positive conditioning probability,*

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

For a discrete unknown X and observed $Y = y$, the corresponding update is

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{\sum_{x'} p(y \mid x')p(x')}. \tag{1.2}$$

The PRIOR $p(x)$ describes uncertainty before observing y. The LIKELIHOOD $p(y \mid x)$ measures how compatible that observation is with a candidate state. The POSTERIOR $p(x \mid y)$ combines them. The denominator is the probability of the evidence and makes the posterior sum to one. Continuous variables use densities and integrals instead of probability masses and sums.

A RANDOM VARIABLE $X : \Omega \to \mathcal{D}$ maps an outcome to a value in its domain $\mathcal{D}$. It is measurable: an event about its value corresponds to an allowed event in Ω. A realization x is one value it takes. For a discrete variable, $p(x) = P(X = x)$; continuous variables use densities and integrals. A RANDOM VECTOR collects several random variables and has a joint distribution over their values.

1.3 Why graphical models?

A joint probability table grows exponentially with the number of variables. A MARGINAL DISTRIBUTION keeps the variables we want by summing out the rest. Without useful structure, this sum also requires exponentially many assignments.

A PROBABILISTIC GRAPHICAL MODEL combines a graph with local functions to describe a joint distribution. The graph says which variables directly participate in each local relationship; the functions give those relationships numerical values. The structure also implies conditional independence statements, which tell us when some information is unnecessary. This gives three related tasks:

1. **Representation:** express a distribution using local, interpretable components.
2. **Inference:** compute marginals, conditional probabilities, or other quantities of interest.
3. **Learning:** estimate the parameters, and sometimes the graph, from data.

Local structure makes the representation explicit and modular, and can make marginalization tractable.

Robot localization and mapping use a DYNAMIC BAYESIAN NETWORK. In figure 3, X_t is pose, U_t is an action, and Z_t is a measurement. The arrows connect successive poses, actions to the next pose, and poses to their measurements.

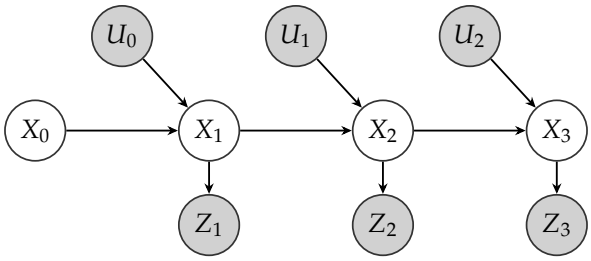


Figure 3: An unrolled localization model. Unshaded nodes are unknown poses; shaded nodes are supplied actions or observed measurements. Action U_t controls the transition from X_t to X_{t+1}.

Other applications have different local structure. Language grounding links phrases to candidate objects and actions through consistency factors. A factorial hidden Markov model uses several latent chains to explain shared observations. Lattice models describe interactions between neighboring spins or pixels, while coding models connect transmitted bits through local constraints. In each case, the graph records which quantities participate in each local relationship.

1.4 Bayesian networks and directed separation

Definition 1.3 [Bayesian network]. A Bayesian network is a directed acyclic graph (DAG) whose nodes index random variables. Each node v has a normalized conditional distribution given its parents $\mathrm{Pa}(v)$. The joint distribution factorizes as

$$p(x_V) = \prod_{v \in V} p(x_v \mid x_{\mathrm{Pa}(v)}). \tag{1.3}$$

A root node has no parents and contributes $p(x_v)$. A parent is simply a node with an arrow into this node; acyclic means that following arrows cannot bring us back to where we started.

A mixture model is a simple example: a component index I points to an observation X, giving $p(i, x) = p(i)p(x \mid i)$. The graph's assumptions reduce the chain rule's conditioning sets to parent sets. The normalized local conditionals make the product a normalized joint distribution.

Definition 1.4 [Conditional independence]. X and Y are conditionally independent given Z, written $X \perp\!\!\!\perp Y \mid Z$, if

$$p(x, y \mid z) = p(x \mid z)p(y \mid z)$$

for conditioning values with positive probability. Equivalently, learning Y provides no additional information about X once Z is known.

The three patterns in figure 4 explain how conditioning changes information flow. In a chain $X \to Z \to Y$, observing Z blocks the path. In a fork $X \leftarrow Z \to Y$, observing the common parent also blocks the path. A collider $X \to Z \leftarrow Y$ behaves differently: the path is blocked when neither

Z nor a descendant of Z is observed. Observing the collider, or one of its descendants, can activate the path.

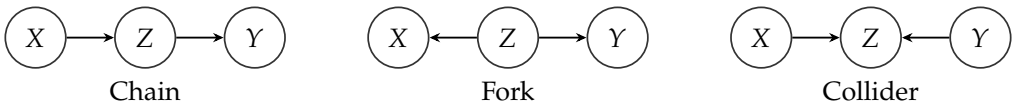


Figure 4: The three local path patterns. Chains and forks are blocked by conditioning on the middle node. A collider is blocked until it or one of its descendants is conditioned on. Arrow directions are essential to this distinction.

For the collider distribution $p(x, y, z) = p(x)p(y)p(z \mid x, y)$, summing over z shows that $p(x, y) = p(x)p(y)$. The root variables are marginally independent. Once z is fixed, however, $p(x, y \mid z)$ is proportional to $p(x)p(y)p(z \mid x, y)$ and need not factor into separate functions of x and y. Observing a shared effect can therefore connect its possible causes.

Definition 1.5 [D-SEPARATION**].** Consider a path in a DAG, ignoring arrow direction when traversing it, and a set of conditioned nodes C. The path is blocked if it contains either

1. a noncollider that belongs to C, or
2. a collider such that neither it nor any of its descendants belongs to C.

Disjoint node sets A and B are D-SEPARATED by C if every path between them is blocked. A distribution that factorizes over the DAG then satisfies $X_A \perp\!\!\!\perp X_B \mid X_C$.

All paths must be blocked to guarantee independence. An active path permits dependence but does not force it. Graphs that imply the same conditional independences are I-EQUIVALENT; chains and forks give an example, since both imply $X \perp\!\!\!\perp Y \mid Z$.

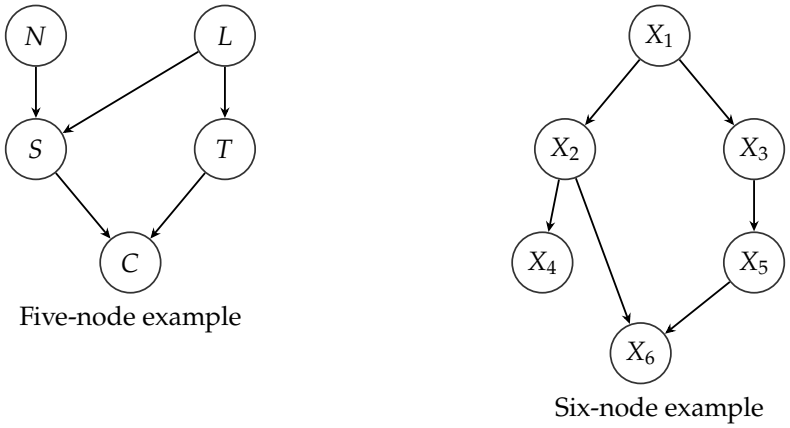


Figure 5: Two Bayesian networks. The five-node graph factors as $p(n)p(l)p(s \mid n, l)p(t \mid l)p(c \mid s, t)$. In the six-node graph, X_6 has parents X_2 and X_5, while every other nonroot node has one parent.

For the six-node graph,

$$p(x_{1:6}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1)p(x_4 \mid x_2)p(x_5 \mid x_3)p(x_6 \mid x_2, x_5).$$

A full joint table grows exponentially with the number of variables. Bayesian-network tables instead grow exponentially with the number of parents per node. With a fixed bound on parents and a fixed number of states, storage grows linearly with the number of nodes.

1.5 Markov random fields and undirected separation

Definition 1.6 [Markov random field]. An undirected graphical model, or Markov random field (MRF), represents a distribution through nonnegative potential functions on cliques:

$$p(x_V) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(x_C), \qquad Z = \sum_{x_V} \prod_{C \in \mathcal{C}} \psi_C(x_C). \tag{1.4}$$

A clique is a set of mutually adjacent nodes. The partition function Z is the sum of the unnormalized weights. Dividing by it makes probabilities sum to one. It must be finite and positive. Continuous models replace the sum by an integral.

Unlike a Bayesian-network conditional table, an individual potential need not sum to one and need not be a probability distribution. It assigns a compatibility weight to each local configuration: a larger value makes that combination more favored when the other factors are held fixed. Multiplying these weights and normalizing produces the joint distribution.

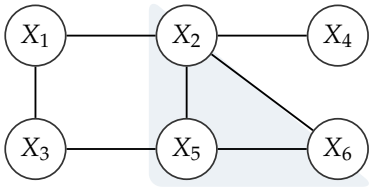


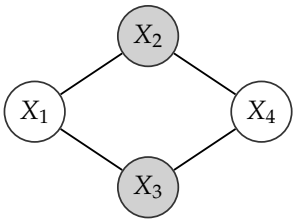
Figure 6: The six-node MRF used below for variable elimination. Four pairwise cliques and the three-node clique $\{2, 5, 6\}$ support the displayed factorization. The shaded triangle highlights the three-variable potential.

For figure 6, use the factorization

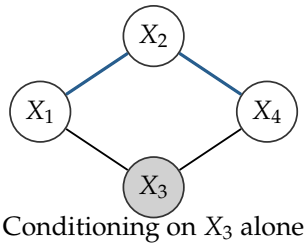
$$p(x_{1:6}) = \frac{1}{Z}\psi_{12}(x_1, x_2)\psi_{13}(x_1, x_3)\psi_{24}(x_2, x_4)\psi_{35}(x_3, x_5)\psi_{256}(x_2, x_5, x_6). \tag{1.5}$$

The triangle contributes the single potential ψ_{256}, which depends on all three variables together.

In an undirected graph, separation has its ordinary graph meaning. A set C separates A and B if every path from A to B passes through C. For a distribution factorized as in equation (1.4), this implies $X_A \perp\!\!\!\perp X_B \mid X_C$.



$X_1 \perp\!\!\!\perp X_4 \mid X_2, X_3$



Conditioning on X_3 alone leaves the upper path open

Figure 7: Undirected separation in a four-cycle. Shading denotes conditioning. Both middle nodes must be conditioned on to block both paths between X_1 and X_4.

1.5.1 Examples of potentials

For discrete variables, a potential can be a table of nonnegative weights. A binary four-cycle has the following example potentials:

$$\psi_{AB} = \psi_{DC} = \begin{pmatrix} 10 & 2 \\ 2 & 10 \end{pmatrix}, \qquad \psi_{AC} = \psi_{BD} = \begin{pmatrix} 2 & 10 \\ 10 & 2 \end{pmatrix}.$$

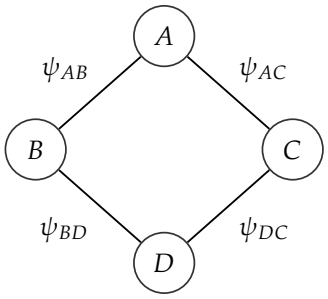
Rows index the first named variable's values $0, 1$, and columns index the second's. The first pair of tables favors agreement; the second favors disagreement. None is individually normalized. Their product defines the joint model

$$p(a, b, c, d) \propto \psi_{AB}(a, b)\psi_{AC}(a, c)\psi_{BD}(b, d)\psi_{DC}(d, c).$$

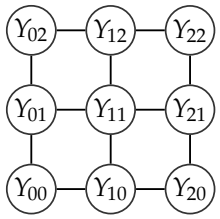
An Ising model uses spins $Y_s \in \{-1, +1\}$ and pair potentials

$$\psi_{st}(y_s, y_t) = \exp(w_{st} y_s y_t).$$

A positive w_{st} favors agreeing spins because agreement contributes $e^{w_{st}}$ rather than $e^{-w_{st}}$. Negative coupling favors disagreement. The factors can be placed on a grid of neighboring sites.



Tabular four-cycle



Ising grid

Figure 8: The tabular model multiplies the four edge potentials. The Ising grid uses pairwise potentials between neighboring spins.

A Gaussian Markov random field uses quadratic single-variable and pairwise potentials. For a symmetric positive-definite matrix Λ and a vector η,

$$p(x) \propto \prod_i \exp\left[-\tfrac{1}{2}\left(\Lambda_{ii} x_i^2 + 2\eta_i x_i\right)\right] \prod_{i<j} \exp(-\Lambda_{ij} x_i x_j).$$

Each undirected pair appears once. The single-variable factors supply quadratic and linear terms; the pairwise factors supply cross terms.

1.6 Converting a Bayesian network to an MRF

A Bayesian-network factor $p(x_v \mid x_{\mathrm{Pa}(v)})$ involves the child and all its parents. To make its scope a clique in an undirected graph, connect every pair of parents of a common child, then remove all arrowheads. This operation is moralization. The parents occur together in the child's conditional table, so they must form a clique with the child. The resulting graph supports the same joint distribution.

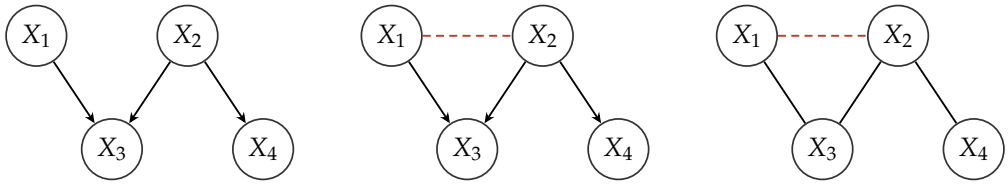


Figure 9: Moralization adds the dashed edge between X_1 and X_2, the parents of X_3, and removes arrowheads. The child and its parents then form the clique required by the conditional factor.

For the example in figure 9,

$$p(x_{1:4}) = p(x_1)p(x_2)p(x_3 \mid x_1, x_2)p(x_4 \mid x_2).$$

Choose $\psi_{123} = p(x_1)p(x_2)p(x_3 \mid x_1, x_2)$ and $\psi_{24} = p(x_4 \mid x_2)$. Their product is the original normalized joint, so $Z = 1$.

1.7 Inference with evidence: from factors to messages

Consider three independent binary events D_1, D_2, D_3 and two observed findings F_1, F_2. The first finding depends on D_1, D_2; the second depends on D_2, D_3. Each finding is a common effect, so observing it can connect its possible causes. Its joint distribution is

$$p(d_1, d_2, d_3, f_1, f_2) = p(d_1)p(d_2)p(d_3)p(f_1 \mid d_1, d_2)p(f_2 \mid d_2, d_3). \tag{1.6}$$

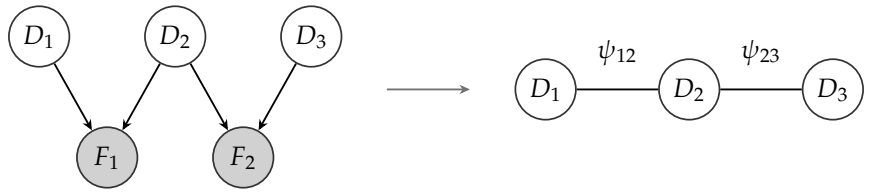


Figure 10: Conditioning on the two findings leaves a chain of event variables. The finding values are absorbed into the pairwise potentials rather than retained as unknown nodes.

Write the fixed evidence as $e = (f_1, f_2)$ and define

$$\begin{aligned} \psi_{12}(d_1, d_2) &= p(d_1)p(d_2)p(f_1 \mid d_1, d_2), \\ \psi_{23}(d_2, d_3) &= p(d_3)p(f_2 \mid d_2, d_3). \end{aligned} \tag{1.7}$$

Then $p(d_1, d_2, d_3, e) = \psi_{12}\psi_{23}$ and

$$p(d_1, d_2, d_3 \mid e) = \frac{\psi_{12}(d_1, d_2)\psi_{23}(d_2, d_3)}{Z_e}, \quad Z_e = \sum_{d_1, d_2, d_3} \psi_{12}\psi_{23} = p(e).$$

The prior $p(d_2)$ appears only once, so it is not double-counted. Conditioning requires $Z_e > 0$.

1.7.1 Computing one marginal

To find $p(d_3 \mid e)$, first compute the unnormalized marginal $p(d_3, e)$:

$$\begin{aligned} p(d_3, e) &= \sum_{d_1, d_2} \psi_{12}(d_1, d_2)\psi_{23}(d_2, d_3) \\ &= \sum_{d_2} \psi_{23}(d_2, d_3) \underbrace{\sum_{d_1} \psi_{12}(d_1, d_2)}_{m_{1\to 2}(d_2)}. \end{aligned} \tag{1.8}$$

The inner sum depends on d_2 but not on d_3. Compute it once and reuse it. Define

$$m_{1\to 2}(d_2) = \sum_{d_1} \psi_{12}(d_1, d_2), \qquad m_{2\to 3}(d_3) = \sum_{d_2} m_{1\to 2}(d_2)\psi_{23}(d_2, d_3).$$

These functions are MESSAGES: each summarizes the contribution of the variables already summed out. The desired posterior is

$$p(d_3 \mid e) = \frac{m_{2\to 3}(d_3)}{\sum_{d_3'} m_{2\to 3}(d_3')}.$$

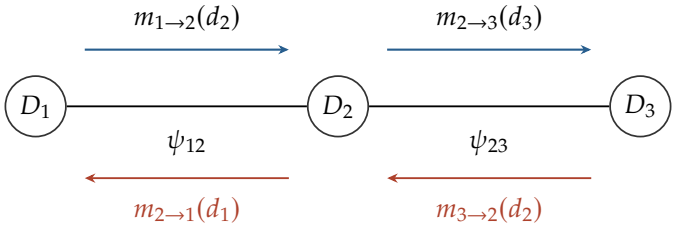


Figure 11: Forward messages in blue and reverse messages in red. Each arrow is labeled by the variable that survives its summation. The two passes provide all three node marginals without repeating the full joint sum.

1.7.2 Reusing work for all marginals

Compute the reverse messages

$$m_{3\to 2}(d_2) = \sum_{d_3} \psi_{23}(d_2, d_3), \qquad m_{2\to 1}(d_1) = \sum_{d_2} \psi_{12}(d_1, d_2)m_{3\to 2}(d_2).$$

Then the unnormalized node marginals are

$$\begin{aligned} p(d_1, e) &= m_{2\to 1}(d_1), \\ p(d_2, e) &= m_{1\to 2}(d_2)m_{3\to 2}(d_2), \\ p(d_3, e) &= m_{2\to 3}(d_3). \end{aligned}$$

Each sums to the same $Z_e = p(e)$, so normalizing any of these vectors gives its posterior marginal.

1.8 Variable elimination in a graph with cycles

Variable elimination multiplies the factors containing the next variable, sums that variable out, and replaces those factors by the resulting function. Other factors remain outside the sum. Distributivity makes this rearrangement exact.

Return to the six-node MRF in equation (1.5). To obtain $p(x_1)$, eliminate X_6, X_5, X_4, X_3, X_2 in that order. Suppress function arguments only when they are apparent from the subscripts.

A message subscript identifies the eliminated variable; its arguments index the remaining variables. Thus $m_6(x_2, x_5)$ is a table, not a scalar.

Algorithm 1.1: Eliminate variables in the six-node MRF

Input: Potentials $\psi_{12}, \psi_{13}, \psi_{24}, \psi_{35}, \psi_{256}$ in equation (1.5)
Output: Marginal $p(x_1)$

$m_6(x_2, x_5) \leftarrow \sum_{x_6} \psi_{256}(x_2, x_5, x_6)$

$m_5(x_2, x_3) \leftarrow \sum_{x_5} \psi_{35}(x_3, x_5) m_6(x_2, x_5)$

$m_4(x_2) \leftarrow \sum_{x_4} \psi_{24}(x_2, x_4)$

$m_3(x_1, x_2) \leftarrow \sum_{x_3} \psi_{13}(x_1, x_3) m_5(x_2, x_3)$

$m_2(x_1) \leftarrow \sum_{x_2} \psi_{12}(x_1, x_2) m_4(x_2) m_3(x_1, x_2)$

$Z \leftarrow \sum_{x_1} m_2(x_1)$

return $p(x_1) = m_2(x_1)/Z$

Each assignment computes a message for every combination of its remaining arguments. Only factors involving the variable being eliminated go inside that sum; factors independent of it can be used in a later step. Only ψ_{256} contains x_6, so the first step replaces it by m_6. The next step combines m_6 with ψ_{35} and removes x_5. Its result $m_5(x_2, x_3)$ couples X_2 and X_3, although the original graph had no edge between them. The remaining rows apply the same rule.

m_2 is the unnormalized marginal; its sum is Z. The nested sums avoid enumerating the full joint table.

1.8.1 Fill-in edges and the induced graph

Eliminating a node generally produces a factor on its remaining neighbors. Make those neighbors into a clique before removing the node. Any newly introduced edge is a FILL-IN EDGE. The graph formed by the original edges and all fill-in edges is the INDUCED GRAPH for that elimination order.

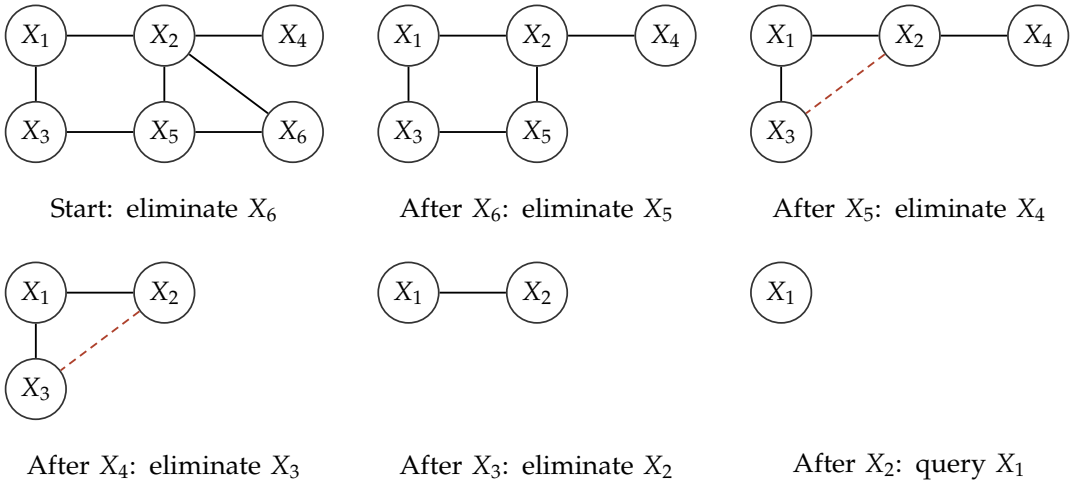


Figure 12: Eliminate $X_6, \ldots, X_2$, keeping nodes at fixed positions. Eliminating X_6 adds no edge because X_2 and X_5 are adjacent. Eliminating X_5 adds the dashed edge X_2–X_3; later steps add none.

1.8.2 Complexity depends on the elimination order

For r states per variable, direct summation over the six-variable joint costs $O(r^6)$. The displayed order uses products involving at most three variables, giving $O(r^3)$ cost for this fixed graph.

Definition 1.7 [TREEWIDTH: CLIQUE-SIZE CONVENTION]. An elimination order's width is the largest clique size encountered in its induced graph. The TREEWIDTH is the minimum of this width over all elimination orders.

The ordering goal is to minimize the largest intermediate clique, because summation cost grows exponentially with its size. Finding an order that achieves treewidth is NP-hard. The ordering problem also has parallels with solving linear equations.

Source. Matthew R. Walter, TTIC 31170, Lecture 1: Introduction & Graphical Models, September 28, 2026.

Lecture 2: Recursive Bayesian Estimation

Recursive estimation infers random states from observations, summarizing the relevant past in the current state and reusing earlier computations.

2.1 Probability spaces and stochastic processes

Definition 2.1 [Probability space and random variable]. A probability space $(\Omega, \mathcal{S}, P)$ consists of an outcome space Ω, a collection $\mathcal{S}$ of measurable events, and a probability measure P. Each event is a subset of Ω; $P(\Omega) = 1$, probabilities are nonnegative, and probabilities of disjoint events add countably. A random variable $X : \Omega \to D$ is a measurable function into a discrete or continuous value space D. In particular,

$$P(X = x) = P\big(\{\omega \in \Omega : X(\omega) = x\}\big).$$

Definition 2.2 [Stochastic process]. A stochastic process is a family of random variables indexed by time or another coordinate. Write $X[n, \omega]$ for discrete time and $X(t, \omega)$ for continuous time. Fixing ω gives a sample path; fixing time gives a random variable across outcomes.

Time and value space are separate choices:

	Discrete values	Continuous values
Discrete time	Discrete random sequence	Discrete random process
Continuous time	Continuous random sequence	Continuous random process

The index need not be time: an image has two spatial indices, and a video has two spatial indices and one temporal index.

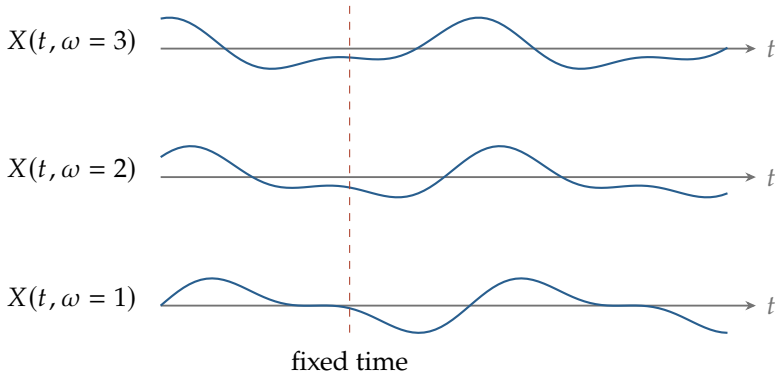


Figure 13: Schematic sample paths. Each row fixes an outcome; the dashed vertical line selects values of one random variable across outcomes. Curves illustrate the distinction and are not measured data.

2.1.1 Examples

For a robot in a discretized one-dimensional environment, $X_t \in \{1, \dots, n\}$ is its cell at time t. Wheel slip and model error make its trajectory uncertain. Write $X^T = (X_0, \dots, X_T)$ for the history through time T; this superscript denotes a history, not a power or transpose.

In language modeling, X_i is the word at position i in an utterance, for example "wreck a nice beach." Such sequences arise in speech recognition and machine translation. In binary communication, $X_t \in \{0, 1\}$ is a transmitted bit in a sequence sent through a noisy channel.

2.2 Means, covariance, and stationarity

For a real vector-valued process with finite second moments, define

$$\begin{aligned} \mu_x(t) &= \mathbb{E}[x(t)], \\ K_{xx}(t, s) &= \mathbb{E}[(x(t) - \mu_x(t))(x(s) - \mu_x(s))^\top], \\ R_{xx}(t, s) &= \mathbb{E}[x(t)x(s)^\top] = K_{xx}(t, s) + \mu_x(t)\mu_x(s)^\top. \end{aligned} \tag{2.1}$$

These are the mean, AUTOCOVARIANCE, and AUTOCORRELATION functions. The discrete-time definitions replace t, s by n, m and parentheses by brackets. Covariance measures joint variation after subtracting the means; autocorrelation retains the mean contribution.

Definition 2.3 [WIDE-SENSE STATIONARITY]. A process is WIDE-SENSE STATIONARY if its mean is constant and its autocorrelation depends only on the lag $\tau = t - s$:

$$\mu_x(t) = \mu, \qquad R_{xx}(t, s) = R_{xx}(t - s).$$

For a real scalar process, $R_{xx}(\tau) = R_{xx}(-\tau)$. For a real vector process, the corresponding identity includes a transpose: $R_{xx}(\tau) = R_{xx}(-\tau)^\top$.

2.2.1 Random walk

Consider the discrete-time random walk

$$x[n] = x[n-1] + w[n], \qquad x[n] = x[0] + \sum_{k=1}^{n} w[k],$$

where $x[0]$ is deterministic and the increments $w[k]$ are independent and identically distributed with mean μ_w and variance σ^2. Equivalently,

$$\mathbb{E}[(w[n] - \mu_w)(w[m] - \mu_w)] = \sigma^2 \delta[n - m],$$

where $\delta[0] = 1$ and $\delta[k] = 0$ for $k \neq 0$ is the discrete unit impulse. Linearity of expectation and independence give

$$\mathbb{E}[x[n]] = x[0] + n\mu_w, \qquad \mathrm{Var}(x[n]) = n\sigma^2. \tag{2.2}$$

The first formula reduces to $n\mu_w$ when $x[0] = 0$. Even with zero drift, a nondegenerate random walk is not wide-sense stationary: its variance grows.

For $n > m$, only the increments after time m remain in the difference:

$$x[n] - x[m] = \sum_{k=m+1}^{n} w[k].$$

Consequently,

$$\mathbb{E}[x[n] - x[m]] = (n - m)\mu_w, \qquad \mathrm{Var}(x[n] - x[m]) = (n - m)\sigma^2.$$

Increments over disjoint intervals are independent, since they use disjoint sets of independent $w[k]$. Also,

$$\mathrm{Cov}(x[n] - x[m], x[k]) = 0 \qquad (n > m \geq k),$$

because $x[k]$ uses only increments at or before k. If the increments were assumed only uncorrelated, these covariance conclusions would hold, but independence would not follow in general.

2.2.2 Valid covariance functions and white noise

A scalar covariance function is symmetric and POSITIVE SEMIDEFINITE: for every finite collection of times $t_1, \dots, t_r$ and real numbers $a_1, \dots, a_r$,

$$\sum_{i,j=1}^{r} a_i K_{xx}(t_i, t_j) a_j \geq 0. \tag{2.3}$$

Conversely, a symmetric function with this property can serve as a covariance function. Positive semidefiniteness constrains the whole matrix of values, rather than requiring every entry to be nonnegative.

Proof. The key idea is that the left side is a variance. Set $z = \sum_i a_i x(t_i)$. Then

$$\operatorname{Var}(z) = \sum_{i,j} a_i \operatorname{Cov}(x(t_i), x(t_j)) a_j \geq 0.$$

For an integrable weighting function, the continuous counterpart is $z = \int a(t)x(t)\,dt$. Whenever these integrals and second moments exist,

$$\operatorname{Var}(z) = \iint a(t) K_{xx}(t, s) a(s)\,dt\,ds \geq 0.$$

This proves the necessary positive-semidefinite condition. □

DISCRETE-TIME WHITE NOISE has uncorrelated samples at distinct times; its means need not be constant. A common zero-mean, constant-variance case has

$$\mathbb{E}[x[n]] = 0, \qquad R_{xx}[n, m] = \sigma^2 \delta[n - m],$$

which is wide-sense stationary. Independent $\mathcal{N}(0, \sigma^2)$ samples give Gaussian white noise. Whiteness alone does not imply independence.

2.3 Least squares estimation

Suppose $x \in \mathbb{R}^n$ is an unknown constant vector and $z \in \mathbb{R}^m$ contains linear, noisy measurements:

$$z_i = \sum_{j=1}^{n} H_{ij} x_j + v_i, \qquad z = Hx + v, \quad H \in \mathbb{R}^{m \times n}.$$

No probability model for the noise is needed to define least squares. Choose the estimate whose predicted measurements are closest to the observed ones.

Definition 2.4 [Least squares]. The residual is $\epsilon_z = z - H\hat{x}$. The least squares estimate minimizes

$$J(\hat{x}) = \epsilon_z^\top \epsilon_z = \|z - H\hat{x}\|_2^2.$$

Expand the quadratic and differentiate with respect to the column vector $\hat{x}$:

$$\begin{aligned} J(\hat{x}) &= z^\top z - 2\hat{x}^\top H^\top z + \hat{x}^\top H^\top H\hat{x}, \\ \nabla_{\hat{x}} J &= -2H^\top z + 2H^\top H\hat{x}. \end{aligned}$$

Setting the gradient to zero gives the normal equations

$$H^\top H\hat{x} = H^\top z. \tag{2.4}$$

If H has full column rank, $H^\top H$ is positive definite and the minimizer is unique:

$$\hat{x} = (H^\top H)^{-1} H^\top z = H^\dagger z. \tag{2.5}$$

Here $H^\dagger = (H^\top H)^{-1} H^\top$ is the pseudoinverse for this full-column-rank case. There must be at least as many measurements as unknowns, but $m \geq n$ alone is insufficient: the columns of H must be linearly independent.

The estimate is linear in z for this fixed linear measurement model. If H is square and invertible, $\hat{x} = H^{-1}z$. For a tall full-column-rank matrix,

$$H^\dagger H = (H^\top H)^{-1} H^\top H = I_n,$$

so $H^\dagger$ is a left inverse. If the noise samples are independent $\mathcal{N}(0, \sigma^2)$, their joint likelihood is proportional to $\exp[-\|z - Hx\|^2/(2\sigma^2)]$, making least squares also the maximum-likelihood estimate under that additional model.

2.3.1 Orthogonality and projection

Let $\mathcal{R}(H)$ denote the column space of H. Every possible predicted measurement Hy lies in this space. The least squares solution chooses the closest such point to z:

$$H\hat{x} = \mathcal{P}_{\mathcal{R}(H)} z, \qquad \mathcal{P}_{\mathcal{R}(H)} = H(H^\top H)^{-1} H^\top.$$

An exact solution $H\hat{x} = z$ exists if and only if $z \in \mathcal{R}(H)$.

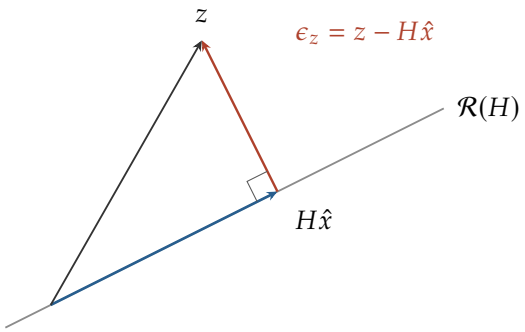


Figure 14: The fitted measurements lie in the column space. The residual is perpendicular to that space, so moving within it cannot reduce the squared error.

Proposition 2.5 [Orthogonality principle]. *The least squares residual satisfies $H^\top \epsilon_z = 0$, and hence $\epsilon_z^\top H y = 0$ for every $y \in \mathbb{R}^n$.*

Proof. The normal equations give $H^\top \epsilon_z = H^\top z - H^\top H \hat{x} = 0$. Therefore $\epsilon_z^\top H y = (H^\top \epsilon_z)^\top y = 0$. Equivalently, substituting the projection explicitly gives

$$z^\top [I - H(H^\top H)^{-1} H^\top] H y = z^\top [H - H I_n] y = 0.$$

□

Example: For repeated measurements of a scalar, $z_i = x + v_i$ for $i = 1, \dots, m$, the matrix H is the $m \times 1$ column of ones. Thus

$$H^\top H = m, \qquad H^\top z = \sum_{i=1}^{m} z_i, \qquad \hat{x} = \frac{1}{m} \sum_{i=1}^{m} z_i.$$

Least squares returns the sample mean.

2.4 The Markov assumption

Without a simplifying assumption, the chain rule gives

$$P(x_{0:T}) = P(x_0) \prod_{t=1}^{T} P(x_t \mid x_{0:t-1}).$$

Each conditional can depend on the entire past.

Definition 2.6 [First-order Markov property]. A process is first-order Markov if, given its present state, its next state is conditionally independent of earlier states:

$$P(X_{t+1} \mid X_t, X_{t-1}, \ldots, X_0) = P(X_{t+1} \mid X_t).$$

Consequently,

$$P(x_{0:T}) = P(x_0) \prod_{t=1}^{T} P(x_t \mid x_{t-1}). \tag{2.6}$$

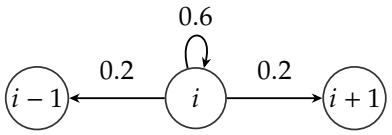
The initial distribution is a separate factor; there is no transition from an unspecified X_{-1}.

For the one-dimensional robot, the transition model at an interior cell is

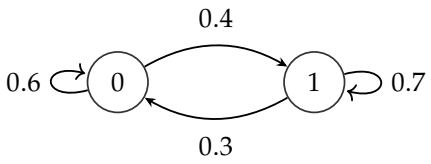
$$P(X_t = j \mid X_{t-1} = i) = \begin{cases} 0.2, & j = i + 1, \\ 0.6, & j = i, \\ 0.2, & j = i - 1, \\ 0, & \text{otherwise.} \end{cases}$$

The boundary cells require their own valid transition probabilities. For the binary sequence, the transition diagram specifies

$$a_{00} = 0.6, \quad a_{10} = 0.4, \quad a_{01} = 0.3, \quad a_{11} = 0.7, \qquad a_{ji} = P(X_t = j \mid X_{t-1} = i).$$



One step from an interior cell



Binary-state Markov chain

Figure 15: The two transition models. Outgoing probabilities from each state sum to one; they describe state evolution rather than sensor errors.

2.5 Hidden Markov models and forward–backward inference

The state often cannot be observed directly. A sensor instead generates a noisy observation, such as $Z_t = X_t + \text{noise}$.

Definition 2.7 [Hidden Markov model]**.** A hidden Markov model (HMM) has a Markov state sequence X_t and observations Z_t that are conditionally independent of other states and observations given X_t. Its joint distribution is

$$P(x_{0:T}, z_{0:T}) = P(x_0)P(z_0 \mid x_0) \prod_{t=1}^{T} P(x_t \mid x_{t-1})P(z_t \mid x_t). \tag{2.7}$$

The two local models are the transition and emission distributions.

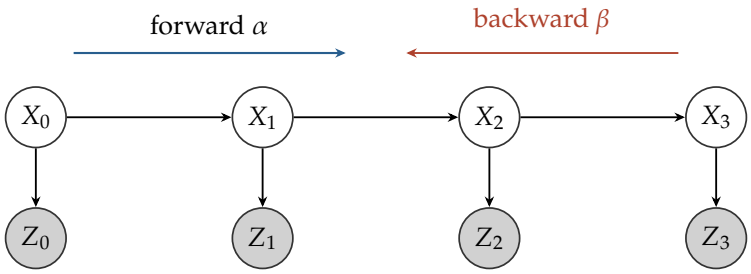


Figure 16: An HMM with observed nodes shaded. Each state emits its observation; forward and backward messages summarize the two sides of the chain.

Fix the observed sequence $z_{0:T}$. To infer one state, use

$$P(X_t = i \mid z_{0:T}) = \frac{P(X_t = i, z_{0:T})}{P(z_{0:T})}.$$

Given X_t, future observations are independent of the earlier observations. This splits the numerator into two reusable parts:

$$\begin{aligned} \alpha_t(i) &= P(z_{0:t}, X_t = i), \\ \beta_t(i) &= P(z_{t+1:T} \mid X_t = i), \\ P(X_t = i, z_{0:T}) &= \alpha_t(i)\beta_t(i). \end{aligned} \tag{2.8}$$

These are the forward and backward probabilities. The forward message includes the observation at time t; the backward message starts at $t + 1$, so that observation is not counted twice.

2.5.1 Forward recursion

Sum over the previous state and apply the HMM independences:

$$\begin{aligned}\alpha_t(i) &= \sum_j P(z_{0:t}, X_{t-1} = j, X_t = i) \\ &= P(z_t \mid X_t = i) \sum_j P(X_t = i \mid X_{t-1} = j) P(z_{0:t-1}, X_{t-1} = j) \\ &= b_i(z_t) \sum_j a_{ij} \alpha_{t-1}(j),\end{aligned}$$

where $b_i(z) = P(Z_t = z \mid X_t = i)$ and $a_{ij} = P(X_t = i \mid X_{t-1} = j)$. Start with

$$\alpha_0(i) = b_i(z_0)\pi_i, \qquad \pi_i = P(X_0 = i).$$

Each update first collects probability arriving from all previous states, then multiplies by how well the new state explains the observation.

For the binary-state example, the first update is explicitly

$$\begin{aligned}\alpha_1(0) &= b_0(z_1)[a_{00}\alpha_0(0) + a_{01}\alpha_0(1)], \\ \alpha_1(1) &= b_1(z_1)[a_{10}\alpha_0(0) + a_{11}\alpha_0(1)],\end{aligned}$$

with $\alpha_0(0) = b_0(z_0)\pi_0$ and $\alpha_0(1) = b_1(z_0)\pi_1$. Both possible predecessors contribute to each sum.

2.5.2 Backward recursion and normalization

For the future observations, sum over the next state:

$$\begin{aligned}\beta_t(i) &= \sum_j P(z_{t+1:T}, X_{t+1} = j \mid X_t = i) \\ &= \sum_j P(X_{t+1} = j \mid X_t = i) P(z_{t+1} \mid X_{t+1} = j) P(z_{t+2:T} \mid X_{t+1} = j) \\ &= \sum_j a_{ji} b_j(z_{t+1}) \beta_{t+1}(j).\end{aligned}$$

Initialize $\beta_T(i) = 1$: there are no remaining observations after T. Work backward from $T - 1$ to 0.

The OBSERVATION-SEQUENCE LIKELIHOOD is the same at every cut of the chain:

$$L = P(z_{0:T}) = \sum_i \alpha_t(i)\beta_t(i) = \sum_i \alpha_T(i). \tag{2.9}$$

The first equality follows by summing $P(X_t = i, z_{0:T})$ over i; the final one uses $\beta_T(i) = 1$. When $L > 0$,

$$P(X_t = i \mid z_{0:T}) = \frac{\alpha_t(i)\beta_t(i)}{L}. \tag{2.10}$$

Algorithm 2.1: Forward–backward inference

Input: Observations $z_{0:T}$, initial probabilities π_i, transitions a_{ji}, emissions b_i
Output: All state posteriors $\gamma_t(i)$ and likelihood L

for each state i **do**
  $\alpha_0(i) \leftarrow \pi_i b_i(z_0)$
for $t \leftarrow 1$ **to** T **do**
  for each state i **do**
    $\alpha_t(i) \leftarrow b_i(z_t) \sum_j a_{ij} \alpha_{t-1}(j)$
$L \leftarrow \sum_i \alpha_T(i)$
for each state i **do**
  $\beta_T(i) \leftarrow 1$
for $t \leftarrow T-1$ **down to** 0 **do**
  for each state i **do**
    $\beta_t(i) \leftarrow \sum_j a_{ji} b_j(z_{t+1}) \beta_{t+1}(j)$
for $t \leftarrow 0$ **to** T **do**
  for each state i **do**
    $\gamma_t(i) \leftarrow \alpha_t(i)\beta_t(i)/L$

For K states and dense transitions, each time step costs $O(K^2)$; both passes together take $O((T+1)K^2)$ time. One running message vector needs $O(K)$ working memory, independent of sequence length. Computing all smoothed marginals with stored forward messages needs $O((T+1)K)$ storage; returning all of them also requires that much output space.

2.6 MAP sequences and the Viterbi algorithm

The most likely state at each separate time is

$$\hat{x}_t = \arg\max_i P(X_t = i \mid z_{0:T}).$$

Putting these marginal choices together need not give the most likely whole trajectory. A state marginal adds up the probabilities of many trajectories; the joint maximum chooses one compatible trajectory:

$$x^*_{0:T} = \arg\max_{x_{0:T}} P(x_{0:T} \mid z_{0:T}) = \arg\max_{x_{0:T}} P(x_{0:T}, z_{0:T}).$$

The denominator $P(z_{0:T})$ is constant with respect to the trajectory. Enumerating all K^{T+1} trajectories is unnecessary.

Definition 2.8 [Viterbi score]. Let $\delta_t(i)$ be the probability of the best path ending in state i at time t, together with the observations so far:

$$\delta_t(i) = \max_{x_{0:t-1}} P(x_{0:t-1}, X_t = i, z_{0:t}).$$

The Markov property makes the last state a sufficient summary of a path for extending it. For a fixed predecessor j, only its best prefix can improve the best path to i: all such prefixes receive the same transition and emission multiplier. Thus

$$\delta_0(i) = \pi_i b_i(z_0), \qquad \delta_t(i) = b_i(z_t) \max_j \{a_{ij}\delta_{t-1}(j)\}. \tag{2.11}$$

Compared with the forward recursion, the sum becomes a maximum. Record the maximizing predecessor as well:

$$\mathrm{Pre}_t(i) = \arg\max_j \{a_{ij}\delta_{t-1}(j)\}.$$

Once the best terminal state is known, these pointers recover the whole path.

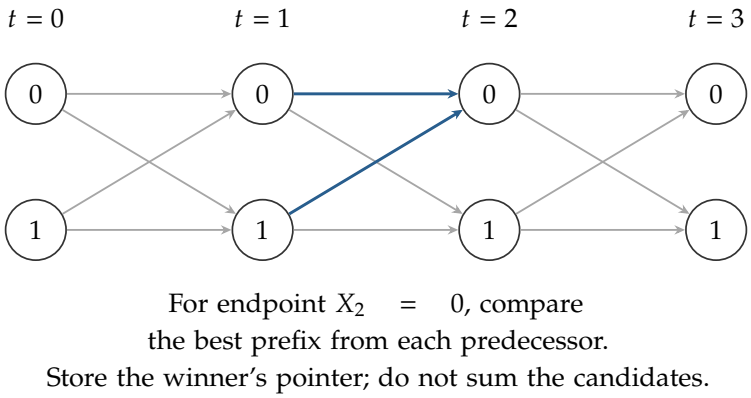


Figure 17: The binary-state trellis. Each column represents one state variable. Blue edges show the candidates for one endpoint; the diagram supplies no numerical emission probabilities and does not select a winner.

Algorithm 2.2: Viterbi decoding

Input: Observations $z_{0:T}$, probabilities π_i, a_{ji}, b_i
Output: A most likely state sequence $x^*_{0:T}$

for each state i **do**
  $\delta_0(i) \leftarrow \pi_i b_i(z_0)$
  $\mathrm{Pre}_0(i) \leftarrow \varnothing$
for $t \leftarrow 1$ **to** T **do**
  for each state i **do**
    $\mathrm{Pre}_t(i) \leftarrow \arg\max_j \{a_{ij}\delta_{t-1}(j)\}$
    $\delta_t(i) \leftarrow b_i(z_t) \max_j \{a_{ij}\delta_{t-1}(j)\}$
$x^*_T \leftarrow \arg\max_i \delta_T(i)$
for $t \leftarrow T-1$ **down to** 0 **do**
  $x^*_t \leftarrow \mathrm{Pre}_{t+1}(x^*_{t+1})$
return $x^*_{0:T}$

Use a consistent choice when maxima tie. As with forward–backward inference, the running time is $O((T+1)K^2)$ for dense transitions. Scores need $O(K)$ working memory, but the backtracking pointers need $O((T+1)K)$ storage. These algorithms assume the observed sequence has positive probability.

Source. Matthew R. Walter, TTIC 31170, Lecture 2: Recursive Bayesian Estimation, September 30, 2026.

Lecture 3: Recursive Bayesian Estimation (Continued)

Filtering estimates the current state; smoothing revisits earlier states using later observations. HMM messages also yield expected counts for learning, while the Bayes filter handles controlled, continuous-state systems.

3.1 Filtering, smoothing, and robot localization

Filtering uses observations only through the time being estimated:

$$P(X_t = i \mid z_{0:t}) = \frac{\alpha_t(i)}{\sum_j \alpha_t(j)}. \tag{3.1}$$

Smoothing also uses later observations. For a fixed history through $T \geq t$,

$$P(X_t = i \mid z_{0:T}) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)} = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_T(j)}. \tag{3.2}$$

The denominators are observation likelihoods, assumed positive. At $t = T$, $\beta_T(i) = 1$, so filtering and smoothing agree. For $t < T$, future observations can change the belief about the state at time t.

Example: A robot moves in a grid using up, down, left, or right actions. Its heading is known. With small probability, the requested motion does not occur. Four sensors report whether there is a wall in front, behind, to the left, and to the right; the illustrated sensor model allows at most one incorrect report.

Initially many cells are plausible. After each requested move, prediction transfers probability to possible destinations, including the possibility of staying put. A sensor update favors cells whose surrounding walls agree with the reports. In the illustrated sequence, several locations remain plausible through the early moves; by time 5, the belief concentrates near the upper junction. A motion or sensor error is a possibility within the model, rather than a reason to discard all earlier information.



Initial ambiguity After repeated updates

Figure 18: The grid geometry with qualitative belief shading. White interior rectangles are obstacles. Darker traversable cells indicate greater belief; the red dots show the illustrated true robot locations. Shading summarizes the initial and final situations and is not a numerical probability reconstruction.

3.2 Learning an HMM from observations

Inference assumes the transition and emission models are known. Learning instead uses observed sequences to estimate these models while the states remain hidden.

Definition 3.1 [Finite HMM parameters]. For finite state space $\mathcal{X}$ and observation alphabet $\mathcal{Z}$, write $\lambda = \{\mathsf{T}, \mathsf{M}, \pi\}$, where

$$\begin{aligned} \mathsf{T}_{ji} &= P(X_{t+1} = j \mid X_t = i; \lambda), \\ \mathsf{M}_{mi} &= P(Z_t = m \mid X_t = i; \lambda), \\ \pi_i &= P(X_0 = i; \lambda). \end{aligned}$$

All entries are nonnegative. Each column of T and M sums to one, and $\sum_i \pi_i = 1$. The source state is the column index of T. These entries agree with the notation a_{ji} and $b_i(m)$.

For observations $z_{0:T}$, maximum-likelihood learning seeks

$$\lambda^* = \arg\max_{\lambda} P(z_{0:T}; \lambda), \qquad P(z_{0:T}; \lambda) = \sum_{x_{0:T}} P(z_{0:T} \mid x_{0:T}; \lambda) P(x_{0:T}; \lambda). \tag{3.3}$$

There are exponentially many state sequences in this written sum. The complete-data joint factorizes locally, but summing out the shared hidden states couples the observations, so one cannot fit an independent model at each time. Forward inference evaluates the likelihood efficiently for a fixed model; it does not solve the global parameter-optimization problem.

The Baum–Welch algorithm uses expectation–maximization (EM): infer expected hidden-state counts under the current model, then fit a new model to those counts. A step seeks λ' with

$$P(z_{0:T}; \lambda') \geq P(z_{0:T}; \lambda).$$

Repeated updates seek a local solution rather than guaranteeing the global maximum. Effective learning needs enough data, often many long observation histories.

3.2.1 Expected state and transition counts

Under the current parameters, define the smoothed state probabilities

$$\gamma_t(i) = P(X_t = i \mid z_{0:T}; \lambda) = \frac{\alpha_t(i)\beta_t(i)}{L}, \qquad L = P(z_{0:T}; \lambda). \tag{3.4}$$

Let $\mathbf{1}\{X_t = i\}$ be one if $X_t = i$ and zero otherwise. Its conditional expectation is $\gamma_t(i)$. Adding indicators and using linearity of expectation gives

$$\begin{aligned} \mathbb{E}[\text{number of visits to } i \mid z_{0:T}] &= \sum_{t=0}^{T} \gamma_t(i), \\ \mathbb{E}[\text{number of departures from } i \mid z_{0:T}] &= \sum_{t=0}^{T-1} \gamma_t(i). \end{aligned} \tag{3.5}$$

The last state contributes a visit, but has no outgoing transition in this observed sequence. This explains the different upper limits.

For adjacent states, define

$$\xi_t(i, j) = P(X_t = i, X_{t+1} = j \mid z_{0:T}; \lambda), \qquad 0 \le t < T. \tag{3.6}$$

Its numerator separates into a prefix, one transition, the next emission, and a suffix:

$$\xi_t(i, j) = \frac{\alpha_t(i)\mathsf{T}_{ji}\mathsf{M}_{z_{t+1},j}\beta_{t+1}(j)}{L}. \tag{3.7}$$

Summing the numerator over i, j gives L, so this is also normalized by its own double sum. In particular,

$$\sum_j \xi_t(i, j) = \gamma_t(i), \qquad \sum_i \xi_t(i, j) = \gamma_{t+1}(j).$$

Thus the expected number of transitions from i to j is $\sum_{t=0}^{T-1} \xi_t(i, j)$.

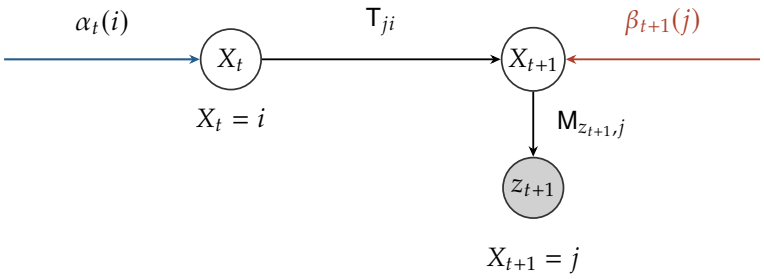


Figure 19: The four factors in the adjacent-state posterior. The next observation belongs to the emission term, and the backward message starts after it.

3.2.2 The Baum–Welch update

Fit the initial distribution, transition probabilities, and emission probabilities using the expected counts:

$$\begin{aligned} \pi'_i &= \gamma_0(i), \\ \mathsf{T}'_{ji} &= \frac{\sum_{t=0}^{T-1} \xi_t(i,j)}{\sum_{t=0}^{T-1} \gamma_t(i)}, \\ \mathsf{M}'_{mi} &= \frac{\sum_{t=0}^{T} \gamma_t(i)\mathbf{1}\{z_t = m\}}{\sum_{t=0}^{T} \gamma_t(i)}. \end{aligned} \tag{3.8}$$

The transition ratio is "expected i-to-j transitions divided by expected departures from i." The emission ratio is "expected visits to i when symbol m was observed divided by all expected visits to i." Summing each ratio over its destination or symbol gives one, using the count identities above. A zero denominator means this sequence supplies no expected count for that column; retaining the current valid column avoids an undefined update.

Algorithm 3.1: One Baum–Welch update for a single sequence

Input: Current model $\lambda = \{\mathsf{T}, \mathsf{M}, \pi\}$ and observations $z_{0:T}$
Output: Updated model λ'

Run forward–backward inference under λ to obtain α, β, L
Compute $\gamma_t(i)$ and $\xi_t(i,j)$ using equations (3.4) and (3.7)
for each state i **do**
  $\pi'_i \leftarrow \gamma_0(i)$
  if $\sum_{t=0}^{T-1} \gamma_t(i) > 0$ **then**
    for each state j **do**
      Update T'_{ji} using equation (3.8)
  else
    Keep $\mathsf{T}'_{\cdot i} \leftarrow \mathsf{T}_{\cdot i}$
  if $\sum_{t=0}^{T} \gamma_t(i) > 0$ **then**
    for each symbol m **do**
      Update M'_{mi} using equation (3.8)
  else
    Keep $\mathsf{M}'_{\cdot i} \leftarrow \mathsf{M}_{\cdot i}$
return $\lambda' = \{\mathsf{T}', \mathsf{M}', \pi'\}$

With exact inference and maximization, EM does not decrease the observation likelihood. At a regular interior fixed point, the likelihood is stationary; this does not establish a global maximum. The posterior counts are computed using the old model throughout one update, then the new model becomes the current model for the next iteration.

3.3 What belongs in the robot's state?

A STATE is a parameterized description of properties of the world that matter for its future. It is COMPLETE if it retains the past information needed for the transition and measurement laws. Bits can be states in communication, words in language models, and robot pose together with relevant environment properties in robotics.

3.3.1 Pose and environment representations

A ground robot's pose is typically (x, y, θ): two Cartesian coordinates and a heading. An aerial or underwater robot typically uses three position coordinates plus roll, pitch, and yaw. A manipulator configuration specifies each joint's angle for a revolute joint or distance for a prismatic joint. Body-frame linear and angular velocities may also belong in the state.

The environment description includes object locations and properties. Walls and trees may be static; people and cars may move. Properties can include appearance, color, or whether a door is open. A map can use continuous parameters, such as wall line segments or landmark poses, or discrete cells, such as an occupancy grid with a probability of occupancy for each cell.

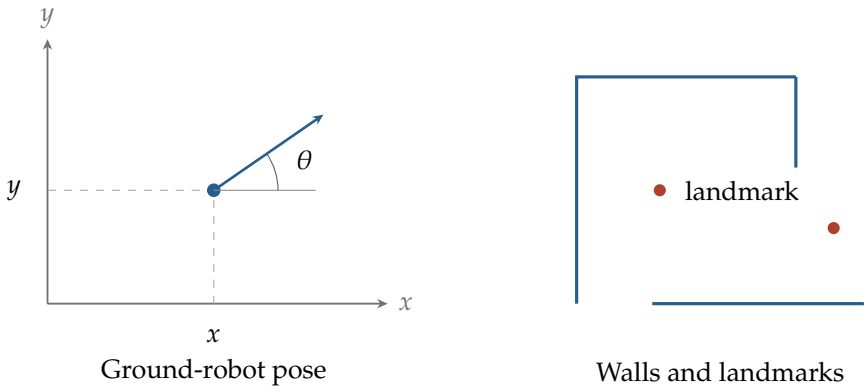


Figure 20: Schematic state representations. Pose gives position and orientation; a feature map gives parameters of surrounding objects. These drawings illustrate representations rather than a particular measured map.

3.3.2 Controls and measurements

A robot interacts with the world in two ways. CONTROL ACTIONS change the world state, for example by moving the robot. SENSOR MEASUREMENTS provide information about it, using a camera, lidar, or tactile sensor. Control data include velocity and odometry; environment measurements include camera images and range data. Bearing-based measurements can report the direction of identifiable landmarks, such as visual fiducials.

Use lowercase notation for the continuous-state setting and write histories as

$$x^t = (x_0, \dots, x_t), \quad u^t = (u_0, \dots, u_t), \quad z^t = (z_0, \dots, z_t).$$

The input u_t describes the control during $(t-1, t]$, while z_t is the measurement at time t.

3.4 Generative laws and belief distributions

The state, control, and measurements vary over time. If the state is complete, the history-dependent models simplify to

$$\begin{aligned} p(x_t \mid x^{t-1}, z^{t-1}, u^t) &= p(x_t \mid x_{t-1}, u_t), \\ p(z_t \mid x_t, z^{t-1}, u^t) &= p(z_t \mid x_t). \end{aligned} \tag{3.9}$$

The first is the STATE TRANSITION MODEL; the second is the MEASUREMENT MODEL. Completeness is a modeling requirement: omitting information needed to predict motion or measurements can invalidate these conditional-independence assumptions.

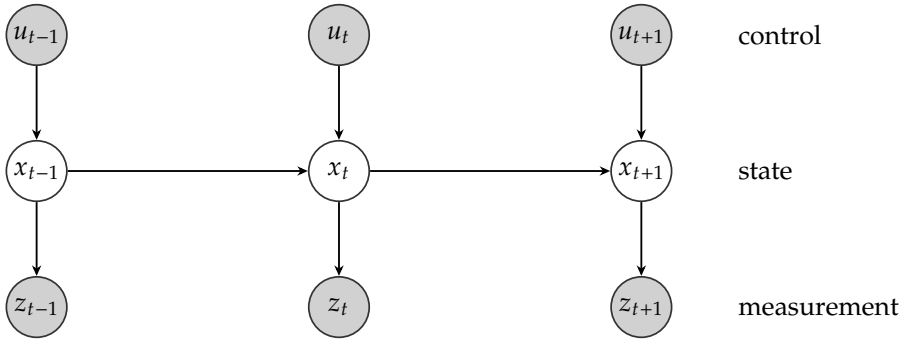


Figure 21: The controlled dynamic Bayesian network. Input u_t affects the transition into x_t; the observation z_t depends on x_t. Known inputs and observed measurements are shaded.

Definition 3.2 [BELIEF AND PREDICTED BELIEF]. The BELIEF is the posterior distribution over the state given all measurements and controls so far:

$$\mathrm{bel}(x_t) = p(x_t \mid z^t, u^t).$$

Before incorporating the current measurement, the PREDICTED BELIEF is

$$\overline{\mathrm{bel}}(x_t) = p(x_t \mid z^{t-1}, u^t).$$

The measurement update converts the predicted belief into the belief.

The prediction includes the latest control and omits the current measurement z_t, while keeping all earlier measurements. The relevant inference tasks are distinguished by the state's time relative to the data history:

Task	State time	Distribution
Filtering	Current	$p(x_t \mid z^t, u^t)$
Prediction	Future, $k > 0$	$p(x_{t+k} \mid z^t, u^t)$
Smoothing	Past, $k < t$	$p(x_k \mid z^t, u^t)$

Future prediction also requires a specified model or plan for future controls.

3.5 Deriving the Bayes filter

Given a prior, a transition model, a measurement model, and a stream of controls and measurements, the Bayes filter recursively computes the current belief. Treat controls as supplied inputs: conditional on the known history, choosing u_t provides no extra observation of x_{t-1}.

3.5.1 Prediction

Marginalize over the unknown previous state:

$$\begin{aligned}\overline{\text{bel}}(x_t) &= \int p(x_t \mid x_{t-1}, z^{t-1}, u^t) p(x_{t-1} \mid z^{t-1}, u^t)\, dx_{t-1} \\ &= \int p(x_t \mid x_{t-1}, u_t)\, \text{bel}(x_{t-1})\, dx_{t-1}.\end{aligned}$$

The transition independence simplifies the first factor. The input convention allows the second factor to be the previous belief. The integral averages possible next-state distributions over uncertainty in the previous state.

3.5.2 Measurement update

Bayes' rule and the measurement independence give

$$\begin{aligned}\text{bel}(x_t) &= \frac{p(z_t \mid x_t, z^{t-1}, u^t) p(x_t \mid z^{t-1}, u^t)}{p(z_t \mid z^{t-1}, u^t)} \\ &= \eta\, p(z_t \mid x_t) \overline{\text{bel}}(x_t),\end{aligned}$$

where the normalizing constant is independent of x_t:

$$\eta^{-1} = \int p(z_t \mid x_t) \overline{\text{bel}}(x_t)\, dx_t.$$

This integral must be finite and positive. Combining the two steps gives

$$\mathrm{bel}(x_t) = \eta\, p(z_t \mid x_t) \int p(x_t \mid x_{t-1}, u_t)\, \mathrm{bel}(x_{t-1})\, dx_{t-1}. \tag{3.10}$$

For discrete states, replace the integral by a sum. The finite-state HMM filter is the same recursion with no explicit control input.

Algorithm 3.2: Bayes filter for a finite state space

Input: Previous belief bel_{t-1}, input u_t, observation z_t, transition and measurement models
Output: Current belief bel_t

for each state x **do**
  $\overline{\mathrm{bel}}_t(x) \leftarrow \sum_{x'} p(x \mid x', u_t)\, \mathrm{bel}_{t-1}(x')$
  $q_t(x) \leftarrow p(z_t \mid x)\overline{\mathrm{bel}}_t(x)$
$c_t \leftarrow \sum_x q_t(x)$
for each state x **do**
  $\mathrm{bel}_t(x) \leftarrow q_t(x)/c_t$
return bel_t

Initialize from $p(x_0)$; if z_0 is available, first normalize $p(z_0 \mid x_0)p(x_0)$. In the discrete procedure, $c_t > 0$ is required and normalization happens after computing weights for all states. A continuous implementation represents and integrates functions instead of enumerating individual real-valued states.

3.6 A hallway with a door sensor

Consider a one-dimensional hallway and a robot with a binary door sensor. The known feature map m gives the door locations; the sensor model is $p(z \mid x, m)$. The map is fixed, so its conditioning can be suppressed in the filter notation.

Initially the belief is uniform because the robot does not know its position. Detecting a door favors three possible door locations, giving three equally plausible hypotheses. There is still positive probability away from doors because the sensor is imperfect. Moving along the hallway shifts the peaks and broadens them to account for motion uncertainty. Another door detection then strengthens the hypothesis consistent with both motion and the door map, while weakening the others. A later motion again shifts and broadens the belief.

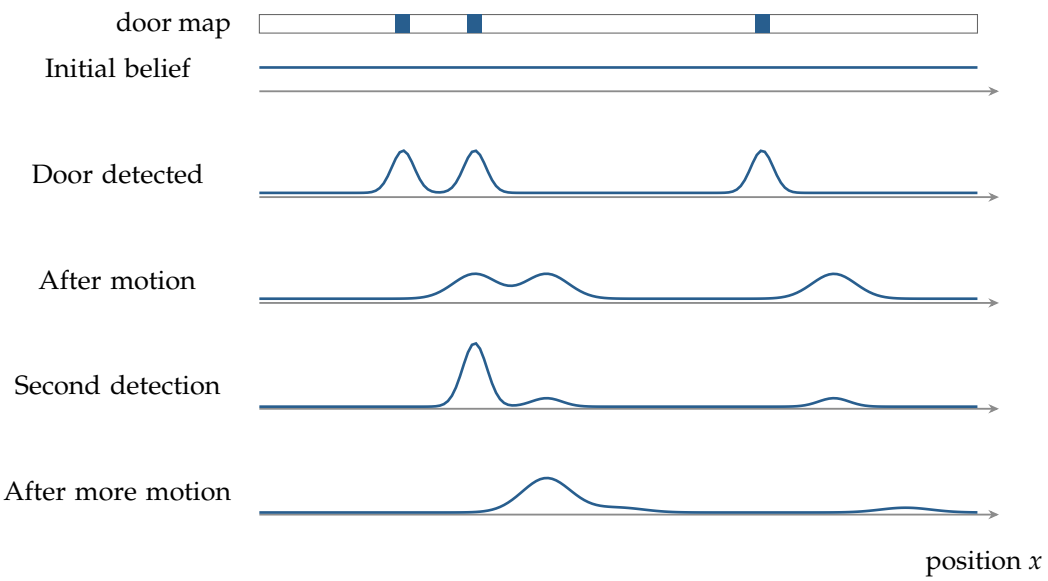


Figure 22: Schematic progression of the hallway example. Rows share a position axis; curve heights are illustrative, with no numerical probability scale. Door observations favor map-compatible positions, and uncertain motion shifts and spreads the belief.

In this example, motion spreads the belief and the observations make it more concentrated. Those are behaviors of the illustrated models and data, rather than universal guarantees that every prediction increases uncertainty or every observation reduces it.

Kalman filters, particle filters, and HMM filters implement this recursion. Belief updates also underlie partially observable Markov decision processes (POMDPs).

Source. Matthew R. Walter, TTIC 31170, Lecture 3: Recursive Bayesian Estimation (Cont.), October 5, 2026.

Lecture 4: Kalman Filters

For linear models with Gaussian noise, Bayes prediction and update preserve Gaussian beliefs, giving the Kalman filter. The extended Kalman filter linearizes nonlinear models around the current estimate.

4.1 The Bayes filter, revisited

The filter is given a stream of measurements and controls, $z^t = (z_0, \dots, z_t)$ and $u^t = (u_0, \dots, u_t)$, a measurement model $p(z_t \mid x_t)$, a transition model $p(x_t \mid x_{t-1}, u_t)$, and a prior $p(x_0)$. It computes the belief $\mathrm{bel}(x_t) = p(x_t \mid z^t, u^t)$. Writing the history out,

$$
\begin{aligned}
\mathrm{bel}(x_t) &= p(x_t \mid u_0, z_0, \dots, u_t, z_t) \\
&\overset{(a)}{=} \eta\, p(z_t \mid x_t, u_0, z_0, \dots, u_t)\, p(x_t \mid u_0, z_0, \dots, u_t) \\
&\overset{(b)}{=} \eta\, p(z_t \mid x_t)\, p(x_t \mid u_0, z_0, \dots, u_t) \\
&\overset{(c)}{=} \eta\, p(z_t \mid x_t) \int p(x_t \mid x_{t-1}, u_0, z_0, \dots, u_t)\, p(x_{t-1} \mid u_0, z_0, \dots, u_t)\, dx_{t-1} \\
&\overset{(d)}{=} \eta\, p(z_t \mid x_t) \int p(x_t \mid x_{t-1}, u_t)\, p(x_{t-1} \mid u_0, z_0, \dots, u_t)\, dx_{t-1} \\
&\overset{(e)}{=} \eta\, p(z_t \mid x_t) \int p(x_t \mid x_{t-1}, u_t)\, \mathrm{bel}(x_{t-1})\, dx_{t-1}.
\end{aligned}
$$

The steps are (a) Bayes' rule, with η the normalizer; (b) conditional independence of z_t from the history given x_t; (c) total probability over x_{t-1}; (d) the Markov property of the transition; and (e) the input convention of equation (3.10): the control u_t carries no information about x_{t-1}, so it can be dropped from the second factor. Splitting the recursion at the integral gives the two steps

$$
\overline{\mathrm{bel}}(x_t) = \int p(x_t \mid x_{t-1}, u_t)\, \mathrm{bel}(x_{t-1})\, dx_{t-1}, \qquad \mathrm{bel}(x_t) = \eta\, p(z_t \mid x_t)\, \overline{\mathrm{bel}}(x_t). \tag{4.1}
$$

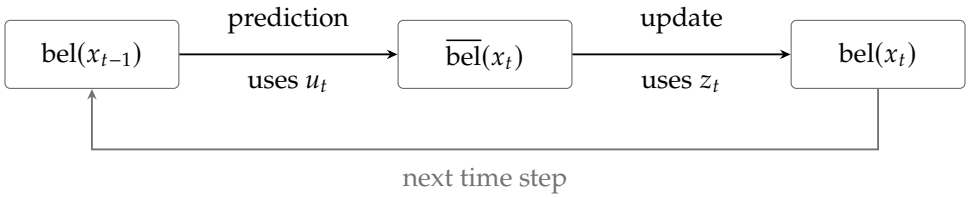


Figure 23: The prediction–update cycle. Prediction pushes the belief through the transition model; the update reweights by the measurement likelihood and normalizes.

4.2 Gaussian distributions

4.2.1 Univariate and multivariate densities

A scalar Gaussian with mean μ and variance σ^2 has density

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma}\exp\!\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}.$$

For $x \in \mathbb{R}^d$ with mean μ and symmetric positive-definite covariance Σ,

$$\mathcal{N}(x;\mu,\Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}}\exp\!\left\{-\tfrac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu)\right\}.$$

The level sets are the ellipsoids $(x-\mu)^\top\Sigma^{-1}(x-\mu) = r^2$. Their axes point along the eigenvectors of Σ, with semi-axis lengths $r\sqrt{\lambda_i}$ for eigenvalues λ_i. Scaling $\Sigma = sI$ makes the contours circles of radius $r\sqrt{s}$: a smaller s gives a narrower, taller peak. For $\Sigma = \left[\begin{smallmatrix} 1 & \rho \\ \rho & 1 \end{smallmatrix}\right]$ the eigenvalues are $1 \pm \rho$ along the diagonal directions, so a larger correlation ρ stretches the contours along $x_1 = x_2$ and squeezes them across it.

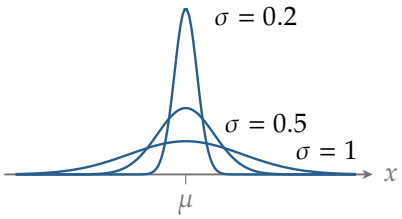


Figure 24: Univariate Gaussian densities with a common mean. All have total area one, so a smaller standard deviation gives a taller and narrower curve.

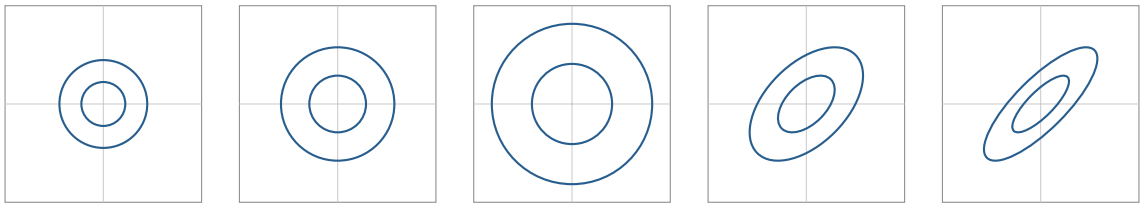


Figure 25: Level sets at Mahalanobis radii 0.75 and 1.5 for zero-mean two-dimensional Gaussians, drawn on a common $[-2.6, 2.6]^2$ window. The first three panels scale an isotropic covariance; the last two introduce positive correlation.

4.2.2 Properties used by the Kalman filter

A Gaussian is unimodal, and its mode, median, and mean coincide. It is completely described by its mean and covariance; an equivalent description by an information vector and information matrix will appear later. Two further properties make Gaussian filtering closed-form.

Proposition 4.1 [Closure under affine maps]. *If $x \sim \mathcal{N}(\mu, \Sigma)$ and $y = Ax + b$, then $y \sim \mathcal{N}(A\mu + b,\ A\Sigma A^\top)$. In one dimension, $y = ax + b$ gives $y \sim \mathcal{N}(a\mu + b,\ a^2\sigma^2)$.*

Proof. An affine map of a Gaussian vector is Gaussian, so it suffices to find the moments. By linearity, $\mathbb{E}[y] = A\mathbb{E}[x] + b = A\mu + b$, and

$$\operatorname{Cov}(y) = \mathbb{E}[(Ax - A\mu)(Ax - A\mu)^\top] = A\Sigma A^\top.$$

If $A\Sigma A^\top$ is singular, y lies in a lower-dimensional affine set and has no density, but the distribution is still the Gaussian with these moments. □

Proposition 4.2 [Product of Gaussian densities]. *For densities in the same variable x,*

$$\mathcal{N}(x; \mu_1, \Sigma_1)\, \mathcal{N}(x; \mu_2, \Sigma_2) \propto \mathcal{N}(x; \mu, \Sigma),$$

where

$$\Sigma = (\Sigma_1^{-1} + \Sigma_2^{-1})^{-1}, \qquad \mu = \Sigma(\Sigma_1^{-1}\mu_1 + \Sigma_2^{-1}\mu_2) = \Sigma_2(\Sigma_1 + \Sigma_2)^{-1}\mu_1 + \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\mu_2.$$

In one dimension,

$$\mu = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2}\mu_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}\mu_2, \qquad \sigma^2 = \frac{1}{\sigma_1^{-2} + \sigma_2^{-2}}.$$

Proof. The exponent of the product is $-\frac{1}{2}$ times

$$(x - \mu_1)^\top \Sigma_1^{-1}(x - \mu_1) + (x - \mu_2)^\top \Sigma_2^{-1}(x - \mu_2) = x^\top \Sigma^{-1} x - 2x^\top(\Sigma_1^{-1}\mu_1 + \Sigma_2^{-1}\mu_2) + c,$$

with c independent of x. Completing the square gives $(x - \mu)^\top \Sigma^{-1}(x - \mu) + c'$ with Σ and μ as stated, so the product is a constant multiple of $\mathcal{N}(x; \mu, \Sigma)$. For the second form of the mean, use $\Sigma = \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\Sigma_2 = \Sigma_2(\Sigma_1 + \Sigma_2)^{-1}\Sigma_1$, which follows from $\Sigma_1^{-1} + \Sigma_2^{-1} = \Sigma_1^{-1}(\Sigma_1 + \Sigma_2)\Sigma_2^{-1}$. Then $\Sigma\Sigma_1^{-1} = \Sigma_2(\Sigma_1 + \Sigma_2)^{-1}$ and $\Sigma\Sigma_2^{-1} = \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}$. □

The product's precision is the sum of the two precisions, and its mean is a precision-weighted average: the more certain factor pulls harder. The product of densities is not the density of the product of two random variables; it is the operation that appears in the Bayes update. Together, the two properties say that beliefs remain Gaussian if one starts with a Gaussian and applies only linear operations.

4.3 The Kalman filter

4.3.1 Linear Gaussian models

Assumption 4.3 (Linear Gaussian system)**.** The process and measurement models are linear in the state and noise:

$$\begin{aligned} x_t &= A_t x_{t-1} + B_t u_t + v_t, & v_t &\sim \mathcal{N}(0, R_t), \\ z_t &= C_t x_t + w_t, & w_t &\sim \mathcal{N}(0, Q_t). \end{aligned}$$

The noise is zero-mean and white, with $\mathbb{E}[v_t v_s^\top] = R_t \delta_{t-s}$ and $\mathbb{E}[w_t w_s^\top] = Q_t \delta_{t-s}$; the two noise sequences are uncorrelated with each other and with x_0. The initial belief is $\mathcal{N}(\mu_0, \Sigma_0)$.

Here $x_t \in \mathbb{R}^n$, $u_t \in \mathbb{R}^l$, and $z_t \in \mathbb{R}^k$. The $n \times n$ matrix A_t describes how the state evolves from $t-1$ to t without control; the $n \times l$ matrix B_t describes how the control u_t changes the state over the same interval; the $k \times n$ matrix C_t maps the state to an observation. Gaussian noise is assumed here for convenience; the zero-mean, white, uncorrelated structure is what the recursion's moment equations use. Under the assumption,

$$p(x_t \mid x_{t-1}, u_t) = \mathcal{N}(x_t; A_t x_{t-1} + B_t u_t,\, R_t), \qquad p(z_t \mid x_t) = \mathcal{N}(z_t; C_t x_t,\, Q_t).$$

The Kalman filter is the Bayes filter for this model. It represents $\mathrm{bel}(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$ and $\overline{\mathrm{bel}}(x_t) = \mathcal{N}(\bar{\mu}_t, \bar{\Sigma}_t)$.

4.3.2 Prediction

Suppose $\mathrm{bel}(x_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$. The prediction integral is

$$\begin{aligned} \overline{\mathrm{bel}}(x_t) &= \int \underbrace{\mathcal{N}(x_t; A_t x_{t-1} + B_t u_t, R_t)}_{p(x_t \mid x_{t-1}, u_t)} \underbrace{\mathcal{N}(x_{t-1}; \mu_{t-1}, \Sigma_{t-1})}_{\mathrm{bel}(x_{t-1})}\, dx_{t-1} \\ &= \eta \int \exp\Big\{-\tfrac{1}{2}(x_t - A_t x_{t-1} - B_t u_t)^\top R_t^{-1} (x_t - A_t x_{t-1} - B_t u_t)\Big\} \\ &\qquad \cdot \exp\Big\{-\tfrac{1}{2}(x_{t-1} - \mu_{t-1})^\top \Sigma_{t-1}^{-1} (x_{t-1} - \mu_{t-1})\Big\}\, dx_{t-1}. \end{aligned}$$

The integrand is the joint density of (x_{t-1}, x_t), and its exponent is quadratic in both. Rather than integrating directly, read the integral as the distribution of $x_t = A_t x_{t-1} + B_t u_t + v_t$ when $x_{t-1} \sim \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$ and v_t is independent of it. The pair (x_{t-1}, v_t) is jointly Gaussian and x_t is affine in it, so x_t is Gaussian with

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \qquad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R_t. \tag{4.2}$$

The cross terms vanish because v_t has zero mean and is uncorrelated with x_{t-1}. This is the A PRIORI ESTIMATE: the motion model moves the mean, maps the covariance through A_t, and adds process noise.

4.3.3 Measurement update

The update multiplies the measurement likelihood by the predicted belief:

$$\begin{aligned}\mathrm{bel}(x_t) &= \eta \underbrace{\mathcal{N}(z_t; C_t x_t, Q_t)}_{p(z_t|x_t)} \underbrace{\mathcal{N}(x_t; \bar{\mu}_t, \bar{\Sigma}_t)}_{\overline{\mathrm{bel}}(x_t)} \\ &= \eta \exp\left\{-\tfrac{1}{2}(z_t - C_t x_t)^\top Q_t^{-1}(z_t - C_t x_t)\right\} \cdot \exp\left\{-\tfrac{1}{2}(x_t - \bar{\mu}_t)^\top \bar{\Sigma}_t^{-1}(x_t - \bar{\mu}_t)\right\}.\end{aligned}$$

Theorem 4.4 [KALMAN UPDATE]. *Assume Q_t and $\bar{\Sigma}_t$ are positive definite. Then* $\mathrm{bel}(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$, *where*

$$K_t = \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}, \qquad \mu_t = \bar{\mu}_t + K_t(z_t - C_t \bar{\mu}_t), \qquad \Sigma_t = (I - K_t C_t)\bar{\Sigma}_t. \tag{4.3}$$

Proof. Expanding the two quadratics, the exponent is $-\frac{1}{2}$ times $x_t^\top \Lambda x_t - 2x_t^\top b_t + c$, with c independent of x_t and

$$\Lambda = C_t^\top Q_t^{-1} C_t + \bar{\Sigma}_t^{-1}, \qquad b_t = C_t^\top Q_t^{-1} z_t + \bar{\Sigma}_t^{-1} \bar{\mu}_t.$$

Since Λ is positive definite, completing the square as in Proposition 4.2 gives a Gaussian with

$$\Sigma_t = \Lambda^{-1}, \qquad \mu_t = \Sigma_t b_t = \Sigma_t (C_t^\top Q_t^{-1} z_t + \bar{\Sigma}_t^{-1} \bar{\mu}_t).$$

Substitute $\bar{\Sigma}_t^{-1} = \Lambda - C_t^\top Q_t^{-1} C_t$ in the mean:

$$\mu_t = \Sigma_t C_t^\top Q_t^{-1} z_t + \bar{\mu}_t - \Sigma_t C_t^\top Q_t^{-1} C_t \bar{\mu}_t = \bar{\mu}_t + \Sigma_t C_t^\top Q_t^{-1}(z_t - C_t \bar{\mu}_t).$$

It remains to identify $\Sigma_t C_t^\top Q_t^{-1}$ with K_t and Λ^{-1} with $(I - K_t C_t)\bar{\Sigma}_t$. Write $S_t = C_t \bar{\Sigma}_t C_t^\top + Q_t$, which is positive definite. Then

$$\Lambda \bar{\Sigma}_t C_t^\top = C_t^\top Q_t^{-1} C_t \bar{\Sigma}_t C_t^\top + C_t^\top = C_t^\top Q_t^{-1} S_t,$$

so $\Lambda K_t = C_t^\top Q_t^{-1}$, that is, $K_t = \Sigma_t C_t^\top Q_t^{-1}$. Using this identity,

$$\Lambda(I - K_t C_t)\bar{\Sigma}_t = \Lambda \bar{\Sigma}_t - C_t^\top Q_t^{-1} C_t \bar{\Sigma}_t = \bar{\Sigma}_t^{-1} \bar{\Sigma}_t = I,$$

so $(I - K_t C_t)\bar{\Sigma}_t = \Lambda^{-1} = \Sigma_t$. □

This is the A POSTERIORI ESTIMATE. The difference $z_t - C_t \bar{\mu}_t$ between the actual and predicted measurement is the INNOVATION, and K_t is the KALMAN GAIN. The gain weighs the predicted

covariance in measurement space, $C_t \bar{\Sigma}_t C_t^\top$, against the measurement noise Q_t: a confident prediction or a noisy sensor gives a small correction, while an uncertain prediction or a precise sensor moves the estimate further toward the measurement. The form of equation (4.3) inverts only the $k \times k$ matrix S_t.

Algorithm 4.1: Kalman filter, one time step

Input: Previous belief $(\mu_{t-1}, \Sigma_{t-1})$, control u_t, measurement z_t
Output: Current belief (μ_t, Σ_t)
// Prediction
$\bar{\mu}_t \leftarrow A_t \mu_{t-1} + B_t u_t$
$\bar{\Sigma}_t \leftarrow A_t \Sigma_{t-1} A_t^\top + R_t$
// Update
$K_t \leftarrow \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}$
$\mu_t \leftarrow \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$
$\Sigma_t \leftarrow (I - K_t C_t) \bar{\Sigma}_t$
return (μ_t, Σ_t)

4.3.4 Properties

The ESTIMATION ERROR $\tilde{x}_t = x_t - \mu_t$ is itself a random variable. Suppose the estimate should minimize a weighted squared error at each step:

$$\mu_t = \arg\min_{\hat{x}} \mathbb{E}\left[(x_t - \hat{x})^\top S \, (x_t - \hat{x}) \;\middle|\; z^t, u^t \right]$$

for a positive-definite weight S.

- If v_t and w_t are zero-mean, uncorrelated, white, and Gaussian, the Kalman filter provides the optimal solution: its mean is the exact conditional mean, which minimizes this criterion.
- If the noise is zero-mean, uncorrelated, and white but not Gaussian, the Kalman filter provides the optimal *linear* solution, the best estimate among those that are linear in the measurements.
- If the noise is correlated or colored, a modified Kalman filter is needed.

The filter is also efficient. With k measurement dimensions and n state dimensions, the cost per step is quoted as $O(k^{2.376} + n^2)$, where the first term is inverting the $k \times k$ matrix S_t with fast matrix inversion; a dense $n \times n$ covariance prediction costs $O(n^3)$ unless A_t has exploitable structure. Memory is $O(n^2)$ for the covariance. The limitation is the linearity assumption: most robotic systems are nonlinear.

The basic filter has several extensions: the extended and unscented Kalman filters for nonlinear systems; sparse information filters for many weakly correlated states; expectation–maximization to estimate the parameters A_t, B_t, C_t, R_t, Q_t; and smoothing, which computes $p(x_t \mid u^T, z^T)$ for $T > t$.

The Kalman filter can also be derived as the recursive least-squares solution to a growing set of linear equations.

Remark 4.5. The filter is named after Rudolf E. Kálmán; with Richard Bucy's continuous-time contribution it is also called the Kalman–Bucy filter. It was used for navigation in the Apollo program.

4.4 The extended Kalman filter

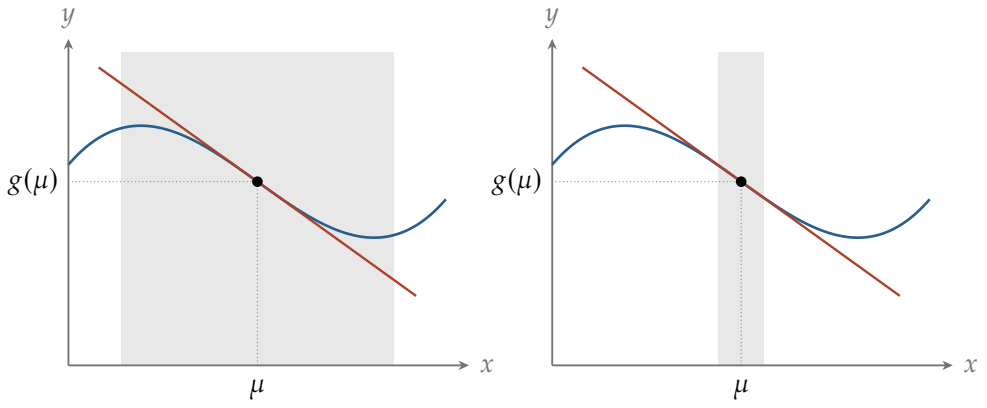
4.4.1 Nonlinear models

Most systems involve nonlinear process and measurement models:

$$x_t = f_t(x_{t-1}, u_t) + v_t, \qquad v_t \sim \mathcal{N}(0, R_t),$$
$$z_t = h_t(x_t) + w_t, \qquad w_t \sim \mathcal{N}(0, Q_t),$$

in place of $A_t x_{t-1} + B_t u_t$ and $C_t x_t$. For an affine map $y = ax + b$ of $x \sim \mathcal{N}(\mu, \sigma^2)$, the output is exactly $\mathcal{N}(a\mu + b, a^2\sigma^2)$. For a nonlinear $y = g(x)$, the output density $p(y)$ is generally not Gaussian; it can be skewed or multimodal, and its mean $\mathbb{E}[g(x)]$ differs from $g(\mu)$.

The *moment-matched* Gaussian shares $p(y)$'s mean and covariance and minimizes $\mathrm{KL}(p \,\|\, q)$ among Gaussian q. Linearization instead uses $g(\mu)$ and the slope at μ. It is accurate when most input mass lies where the tangent is a good approximation; high input variance can violate this condition.



High input variance Low input variance

Figure 26: The same nonlinear function g (blue) and its tangent at the input mean (red). The shaded band is $\mu \pm 2\sigma$. With high variance, g departs far from its tangent inside the band, so the propagated distribution is distorted; with low variance, the tangent is accurate across the band. The curve is illustrative.

4.4.2 Linearization by Taylor expansion

The EXTENDED KALMAN FILTER (EKF) replaces each nonlinear model by its first-order Taylor expansion about the current estimate. For x_{t-1} close to μ_{t-1},

$$f_t(x_{t-1}, u_t) \approx f_t(\mu_{t-1}, u_t) + \left.\frac{\partial f_t(x_{t-1}, u_t)}{\partial x_{t-1}}\right|_{\mu_{t-1}} (x_{t-1} - \mu_{t-1}) = f_t(\mu_{t-1}, u_t) + F_t(x_{t-1} - \mu_{t-1}),$$

and for x_t close to the predicted mean $\bar{\mu}_t$,

$$h_t(x_t) \approx h_t(\bar{\mu}_t) + \left.\frac{\partial h_t(x_t)}{\partial x_t}\right|_{\bar{\mu}_t} (x_t - \bar{\mu}_t) = h_t(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t).$$

The Jacobians F_t ($n \times n$) and H_t ($k \times n$) play the roles of A_t and C_t.

With the linearized process model, x_t is affine in (x_{t-1}, v_t). The affine-closure argument behind equation (4.2) then gives mean $f_t(\mu_{t-1}, u_t)$ and covariance $F_t \Sigma_{t-1} F_t^\top + R_t$. With the linearized measurement model,

$$z_t - h_t(\bar{\mu}_t) + H_t \bar{\mu}_t \approx H_t x_t + w_t,$$

which is a linear measurement of x_t with matrix H_t. Applying theorem 4.4 to this shifted measurement gives the innovation $\big(z_t - h_t(\bar{\mu}_t) + H_t \bar{\mu}_t\big) - H_t \bar{\mu}_t = z_t - h_t(\bar{\mu}_t)$.

Algorithm 4.2: Extended Kalman filter, one time step

Input: Previous belief $(\mu_{t-1}, \Sigma_{t-1})$, control u_t, measurement z_t
Output: Current belief (μ_t, Σ_t)
// Prediction
$F_t \leftarrow \partial f_t(x_{t-1}, u_t)/\partial x_{t-1}$ evaluated at μ_{t-1}
$\bar{\mu}_t \leftarrow f_t(\mu_{t-1}, u_t)$
$\bar{\Sigma}_t \leftarrow F_t \Sigma_{t-1} F_t^\top + R_t$
// Update
$H_t \leftarrow \partial h_t(x_t)/\partial x_t$ evaluated at $\bar{\mu}_t$
$K_t \leftarrow \bar{\Sigma}_t H_t^\top (H_t \bar{\Sigma}_t H_t^\top + Q_t)^{-1}$
$\mu_t \leftarrow \bar{\mu}_t + K_t\big(z_t - h_t(\bar{\mu}_t)\big)$
$\Sigma_t \leftarrow (I - K_t H_t)\bar{\Sigma}_t$
return (μ_t, Σ_t)

The means use the nonlinear functions themselves; only the covariances and gain use the Jacobians.

4.4.3 Properties and extensions

The EKF is not optimal: its Gaussian is a linearization, not the true posterior or even its moment-matched Gaussian. It keeps the Kalman filter's efficiency, with the same quoted cost $O(k^{2.376} + n^2)$ and $O(n^2)$ memory. It can *diverge* if the nonlinearities are large or the uncertainty is high, since the

expansion point may then be far from much of the probability mass. In practice, it often works well even when its assumptions are violated.

For highly nonlinear models, the UNSCENTED KALMAN FILTER applies the nonlinear function to several deterministically chosen sample points (the unscented transform) to better estimate the output mean and covariance. Jacobians can also be computed by numerical linearization. A further question is OBSERVABILITY, the dual of controllability: whether the measurements contain enough information to determine the state that the (E)KF estimates.

Source. Matthew R. Walter, TTIC 31170, Lecture 4: Kalman Filters, October 7, 2026.

Lecture 5: Nonparametric Filters

The Kalman filter needs linear or linearized motion and measurement models, can diverge under significant nonlinearity, and cannot represent multimodal beliefs. Nonparametric filters drop the Gaussian form. The discrete Bayes filter keeps a probability for each cell of a partition of the state space; the particle filter represents the belief by weighted random samples.

5.1 The discrete Bayes filter

Partition the state space $\mathcal{X}$ into disjoint regions $x_{k,t}$:

$$x_{i,t} \cap x_{j,t} = \varnothing \quad \text{for all } i \neq j, \qquad \bigcup_k x_{k,t} = \mathcal{X}.$$

The discrete Bayes filter, or histogram filter, keeps a probability $P_{k,t}$ for each region and spreads it uniformly across the region, so that inside region k

$$p(x_t) = \frac{P_{k,t}}{|x_{k,t}|}.$$

The belief is thus a histogram approximation of the continuous density. Passing a random variable through a nonlinear function $y = g(x)$ maps each bin of the histogram of x to a bin of the histogram of y, and the result approximates the true output density however non-Gaussian it is.

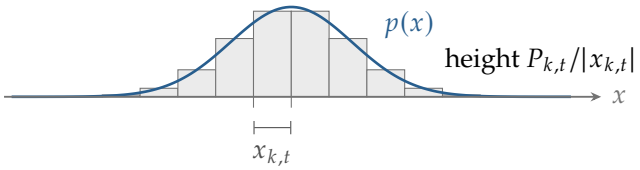


Figure 27: A histogram belief. Each region holds probability $P_{k,t}$, spread evenly over the region, so the bar height is $P_{k,t}/|x_{k,t}|$. The curve is illustrative.

In the hallway example of figure 22, the histogram filter starts from a uniform belief, multiplies by the door likelihood $p(z \mid x)$ to get three peaks, shifts and blurs them when the robot moves, and after a second door detection concentrates most of the mass on a single cell.

Replacing the integral in equation (3.10) by a sum over regions gives the algorithm. It is the forward step of the HMM forward–backward algorithm, with the transition probabilities now depending on u_t.

Algorithm 5.1: Discrete Bayes filter

Input: Previous probabilities $\{P_{k,t-1}\}$, control u_t, measurement z_t
Output: Current probabilities $\{P_{k,t}\}$

$\eta \leftarrow 0$
for each region k **do**
  $\bar{P}_{k,t} \leftarrow \sum_i p(X_t = x_k \mid X_{t-1} = x_i, u_t)\, P_{i,t-1}$
for each region k **do**
  $P_{k,t} \leftarrow p(z_t \mid X_t = x_k)\, \bar{P}_{k,t}$
  $\eta \leftarrow \eta + P_{k,t}$
for each region k **do**
  $P_{k,t} \leftarrow \eta^{-1} P_{k,t}$
return $\{P_{k,t}\}$

Here η accumulates the unnormalized total, so the final loop divides by it. The drawbacks are size and cost. The number of cells n is exponential in the number of state dimensions: a planar pose (x, y, θ) already needs a three-dimensional grid, and accuracy often requires fine resolution. The prediction sums over all n previous cells for each of the n cells, so an update costs $O(n^2)$, and memory for a dense transition table is also $O(n^2)$.

5.2 Monte Carlo approximation

The particle filter keeps a set of samples instead. It can use the actual motion and measurement models, can compute properties of the belief such as its moments, and concentrates computation where the probability is, giving an adaptive resolution. It is also called the Monte Carlo filter, survival of the fittest, or condensation. In global localization the samples start spread over the whole free space of the map and, as measurements arrive, collapse onto a small cluster at the robot's pose.

5.2.1 Empirical distributions

The Dirac delta $\delta(x)$ is a generalized function that vanishes for $x \neq 0$ and satisfies $\int_{-\infty}^{\infty} \delta(x)\, dx = 1$. The Dirac delta distribution $\delta_\mu(x) = \delta(x - \mu)$ puts all mass at μ. Given samples $x^{(i)} \sim p(x)$, the empirical distribution is

$$\hat{p}(x) = \frac{1}{n} \sum_{i=1}^{n} \delta\left(x - x^{(i)}\right).$$

The more samples fall into an interval, the higher the probability assigned to it.

5.2.2 Monte Carlo estimates

For any function f, such as cell occupancy or distance to an obstacle, substituting $\hat{p}$ for p gives

$$\mathbb{E}[f(x)] = \int f(x)p(x)\,dx \approx \int f(x)\,\frac{1}{n}\sum_{i=1}^{n}\delta\big(x - x^{(i)}\big)\,dx = \frac{1}{n}\sum_{i=1}^{n} f\big(x^{(i)}\big) =: \hat{\mathbb{E}}[f(x)].$$

Each term has mean $\mathbb{E}[f(x)]$, so the estimate is unbiased. For independent samples its variance is $\mathrm{Var}(f(x))/n$, inversely proportional to n, and by the strong law of large numbers it converges almost surely to $\mathbb{E}[f(x)]$ as $n \to \infty$.

5.3 Sampling from distributions

Sampling from a Gaussian or a uniform distribution is easy. The general strategy is to reduce to sampling $s \sim \mathrm{Uniform}[0, 1]$.

- *Discrete distributions.* If $X \in \{X_1, \ldots, X_m\}$ with $P(X_i) = \theta_i$, partition $[0, 1]$ into consecutive intervals $[0, \theta_1), [\theta_1, \theta_1 + \theta_2), \ldots$ of lengths θ_i and return X_i when s lands in the ith interval.
- *Inverse transform.* For continuous $x \sim p(x)$ with cumulative distribution $F_x(y) = p(x \le y) = \int_{-\infty}^{y} p(x)\,dx$, return $F_x^{-1}(s)$. This has the right law because $p(F_x^{-1}(s) \le y) = p(s \le F_x(y)) = F_x(y)$. For the exponential density $p(x) = \lambda e^{-\lambda x}$, $F_x(y) = 1 - e^{-\lambda y}$ and $F_x^{-1}(s) = -\lambda^{-1}\ln(1 - s)$. The catch is that F_x often cannot be computed or inverted.
- *Gaussians (polar Box–Muller).* Draw $s_1, s_2 \sim \mathrm{Uniform}[-1, 1]$ independently and reject the pair if it falls outside the unit circle, so that (s_1, s_2) is uniform on the disk with density $1/\pi$. With $r^2 = s_1^2 + s_2^2$,
$$x_1 = s_1\sqrt{\frac{-2\ln r^2}{r^2}}, \qquad x_2 = s_2\sqrt{\frac{-2\ln r^2}{r^2}}$$
are independent $\mathcal{N}(0, 1)$ samples.
- *Rejection sampling.* Suppose $p(x) < 1$ for all x on a bounded domain. Sample x uniformly, sample $c \sim \mathrm{Uniform}[0, 1]$, and keep x if $p(x) > c$. A point is kept with probability $p(x)$, so accepted points have density proportional to p. Points where p is small are mostly rejected, which wastes samples.

5.4 Importance sampling

Suppose $p(x)$ is hard to sample from but easy to evaluate at any point. Take any density q with $q(x) > 0$ wherever $p(x) > 0$. Then

$$\mathbb{E}_{X\sim p}[f(X)] = \int f(x)\,\frac{p(x)}{q(x)}\,q(x)\,dx = \mathbb{E}_{X\sim q}\Big[\frac{p(X)}{q(X)}f(X)\Big] \approx \frac{1}{m}\sum_{i=1}^{m}\frac{p(x^{(i)})}{q(x^{(i)})}f\big(x^{(i)}\big), \qquad x^{(i)} \sim q(x).$$

Equivalently, the weighted empirical distribution

$$\hat{p}(x) = \frac{1}{n}\sum_{i=1}^{n}\frac{p(x^{(i)})}{q(x^{(i)})}\,\delta_{x^{(i)}}(x)$$

approximates p. The TARGET p is hard to sample; the PROPOSAL q is easy to sample; the IMPORTANCE WEIGHT $w^{(i)} = p(x^{(i)})/q(x^{(i)})$ accounts for the difference between them. In a filter, $p(x^{(i)})$ is usually a likelihood, so the weights are likelihoods of the samples.

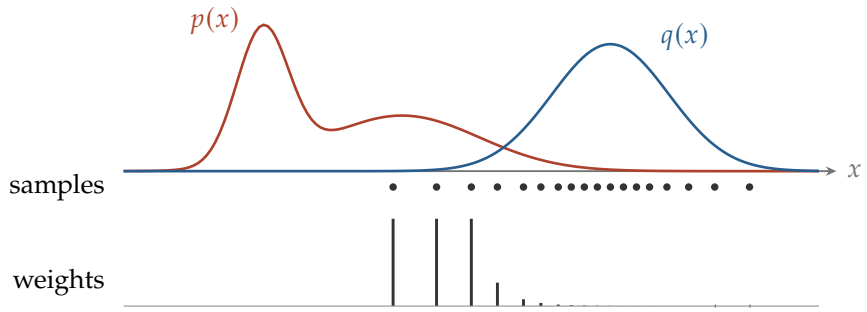


Figure 28: Importance sampling. Samples (dots) are drawn from the proposal q (blue), and each is weighted by p/q (bar heights, capped for display). Samples where the target p (red) is large relative to q get large weights. Here q covers p's left mode poorly, so few samples represent it. Curves are illustrative.

5.4.1 Resampling

IMPORTANCE SAMPLING WITH RESAMPLING converts weighted samples into unweighted ones: sample $x^{(1)}, \ldots, x^{(n)}$ from an easy density such as a uniform, attach the weights $w^{(i)} = p(x^{(i)})/q(x^{(i)})$, and draw n new particles with replacement from $\{x^{(i)}\}$ with probabilities proportional to $\{w^{(i)}\}$. The result is approximately distributed according to the target: high-weight particles are duplicated and low-weight ones disappear.

5.5 Sequential importance sampling

5.5.1 Weights over time

The goal is an importance sampler that approximates $p(x_{1:t})$ sequentially. Suppose $\{x_1^{(i)}\}$ was drawn from $q_1(x_1)$ with weights $w_1^{(i)} = p(x_1^{(i)})/q_1(x_1^{(i)})$. At $t = 2$, extend each sample by $x_2^{(i)} \sim q(x_2 \mid x_1^{(i)})$, so that q factors over time, $q_2(x_{1:2}) = q(x_2 \mid x_1)q(x_1)$. The weight is then

$$w_2 = \frac{p(x_{1:2})}{q(x_2 \mid x_1)q(x_1)} = \frac{p(x_1)}{q(x_1)} \cdot \frac{p(x_{1:2})}{p(x_1)q(x_2 \mid x_1)} = w_1(x_1)\,\frac{p(x_{1:2})}{p(x_1)q(x_2 \mid x_1)}.$$

In general, with $x_t^{(i)} \sim q(x_t \mid x_{t-1}^{(i)}) \cdots q(x_2^{(i)} \mid x_1^{(i)})q(x_1^{(i)})$,

$$w_t = w_{t-1}(x_{1:t-1})\,\frac{p(x_{1:t})}{p(x_{1:t-1})\,q(x_t \mid x_{1:t-1})}.$$

Old samples are never revised: each step appends a new state and multiplies in one factor.

5.5.2 Application to Bayesian filtering

Maintain a belief over the state history $x_{0:t} = (x_0, \ldots, x_t)$ rather than only the current state. Bayes' rule and the Markov assumptions give

$$\mathrm{bel}(x_{0:t}) = p(x_{0:t} \mid u_{1:t}, z_{1:t}) = p(x_{0:t-1} \mid u_{1:t-1}, z_{1:t-1})\,\frac{p(z_t \mid x_t)\,p(x_t \mid x_{t-1}, u_t)}{p(z_t \mid z_{1:t-1})}.$$

For nonlinear or non-Gaussian models this has no closed form, and it is often too complex to sample from directly. Even if it could be sampled, the cost would grow at least linearly in t. The aim is fixed cost per step, without modifying previously sampled trajectories $\{x_{0:t-1}^{(i)}\}_{i=1}^n$. The controls are dropped from the notation below.

Choose a proposal that factors over time:

$$q(x_{0:t} \mid z_{1:t}) = q(x_{0:t-1} \mid z_{1:t-1})\,q(x_t \mid x_{0:t-1}, z_{1:t}) = q(x_0)\prod_{k=1}^{t} q(x_k \mid x_{0:k-1}, z_{1:k}).$$

This is a restriction: the proposal for earlier states cannot use later measurements. The importance weight is

$$w_t(x_{0:t}) = \frac{p(x_{0:t} \mid z_{1:t})}{q(x_{0:t} \mid z_{1:t})} = \frac{1}{p(z_{1:t})}\,\frac{p(x_{0:t}, z_{1:t})}{q(x_{0:t} \mid z_{1:t})}.$$

The factor $p(z_{1:t})$ is the same for every sample and is removed by normalization, so it suffices to track the UNNORMALIZED WEIGHT $\bar{w}_t(x_{0:t}) = p(x_{0:t}, z_{1:t})/q(x_{0:t} \mid z_{1:t})$.

Proposition 5.1 [Weight recursion].

$$\bar{w}_t(x_{0:t}) = \bar{w}_{t-1}(x_{0:t-1}) \frac{p(z_t \mid x_t)\, p(x_t \mid x_{t-1})}{q(x_t \mid x_{0:t-1}, z_{1:t})}.$$

Proof. By the Markov and measurement assumptions, $p(x_{0:t}, z_{1:t}) = p(z_t \mid x_t)\, p(x_t \mid x_{t-1})\, p(x_{0:t-1}, z_{1:t-1})$. Divide by the factored proposal and group $p(x_{0:t-1}, z_{1:t-1})/q(x_{0:t-1} \mid z_{1:t-1}) = \bar{w}_{t-1}(x_{0:t-1})$. □

Algorithm 5.2: Sequential importance sampling

```
for t = 0, 1, 2, ... do
    for each i ∈ {1, ..., n} do
        Sample x_t^(i) ~ q(x_t | x_{0:t-1}^(i), z_{1:t}) and set x_{0:t}^(i) = (x_{0:t-1}^(i), x_t^(i))
        w̄_t^(i) ← w̄_{t-1}^(i) · p(z_t | x_t^(i)) p(x_t^(i) | x_{t-1}^(i)) / q(x_t^(i) | x_{0:t-1}^(i), z_{1:t})
    for each i ∈ {1, ..., n} do
        w_t^(i) ← w̄_t^(i) / Σ_{j=1}^n w̄_t^(j)
```

Each step moves every particle by the proposal and reweights it by how well it explains the new measurement. The particle-filter objective is exactly this: given samples $\mathcal{P}_{t-1} = \{x_{0:t-1}^{(1)}, \ldots, x_{0:t-1}^{(n)}\}$ representing $\mathrm{bel}(x_{0:t-1}) = p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t-1})$, find samples $\mathcal{P}_t = \{x_{0:t}^{(1)}, \ldots, x_{0:t}^{(n)}\}$ representing $\mathrm{bel}(x_{0:t})$.

5.6 Choosing the proposal

The variance of the importance weights increases over time, so the filter diverges: eventually one particle carries nearly all the weight. There are two remedies. The first is to choose a proposal that minimizes the variance of the weights conditioned on $x_{0:t-1}^{(i)}$ and $z_{1:t}$; the second, resampling, is treated in the next subsection.

5.6.1 The optimal proposal

The proposal

$$q\left(x_t \mid x_{0:t-1}^{(i)}, z_{1:t}, u_t\right) = p\left(x_t \mid x_{t-1} = x_{t-1}^{(i)}, z_t, u_t\right)$$

achieves zero conditional variance. By Proposition 5.1 and $p(z_t \mid x_t)\, p(x_t \mid x_{t-1}, u_t) = p(z_t, x_t \mid x_{t-1}, u_t) = p(x_t \mid x_{t-1}, z_t, u_t)\, p(z_t \mid x_{t-1}, u_t)$,

$$\bar{w}_t^{(i)} = \bar{w}_{t-1}^{(i)} \frac{p(z_t \mid x_t^{(i)})\, p(x_t^{(i)} \mid x_{t-1}^{(i)}, u_t)}{p(x_t^{(i)} \mid x_{t-1}^{(i)}, z_t, u_t)} = \bar{w}_{t-1}^{(i)}\, p\left(z_t \mid x_{t-1}^{(i)}, u_t\right) = \bar{w}_{t-1}^{(i)} \int p(z_t \mid x_t)\, p\left(x_t \mid x_{t-1}^{(i)}, u_t\right) dx_t.$$

The new weight does not depend on the sampled $x_t^{(i)}$, so given the past it is deterministic. Two challenges remain: it is often difficult to sample from $p(x_t \mid x_{t-1}, z_t, u_t)$, and the integral for the weight is expensive.

Example: Take nonlinear Gaussian motion with a linear Gaussian measurement,

$$x_t = f_t(x_{t-1}, u_t) + v_t, \quad v_t \sim \mathcal{N}(0, R_t), \qquad z_t = Hx_t + w_t, \quad w_t \sim \mathcal{N}(0, Q_t).$$

Given $x_{t-1}^{(i)}$, the prior on x_t is $\mathcal{N}(f_t(x_{t-1}^{(i)}, u_t), R_t)$, and completing the square as in the proof of theorem 4.4 gives $p(x_t \mid x_{t-1}^{(i)}, u_t, z_t) = \mathcal{N}(m_t, \Sigma)$ with

$$\Sigma = \left(R_t^{-1} + H^\top Q_t^{-1} H\right)^{-1}, \qquad m_t = \Sigma\left(R_t^{-1} f_t(x_{t-1}^{(i)}, u_t) + H^\top Q_t^{-1} z_t\right).$$

Since $z_t = Hx_t + w_t$ is affine in Gaussian variables, $z_t \mid x_{t-1}, u_t \sim \mathcal{N}(Hf_t(x_{t-1}, u_t),\, HR_tH^\top + Q_t)$, so

$$p(z_t \mid x_{t-1}, u_t) \propto \exp\!\Big(-\tfrac{1}{2}\big(z_t - Hf_t(x_{t-1}, u_t)\big)^\top \big(Q_t + HR_tH^\top\big)^{-1}\big(z_t - Hf_t(x_{t-1}, u_t)\big)\Big).$$

Both the optimal proposal and its weight are available in closed form here.

5.6.2 The Markov prior as proposal

The common alternative proposes from the motion model, $q(x_t \mid x_{0:t-1}^{(i)}, z_{1:t}, u_t) = p(x_t \mid x_{t-1} = x_{t-1}^{(i)}, u_t)$. The transition density cancels in Proposition 5.1, leaving

$$\bar{w}_t^{(i)} = \bar{w}_{t-1}^{(i)}\, p\big(z_t \mid x_t^{(i)}\big).$$

This is the STANDARD PARTICLE FILTER. In robotics the proposal is typically a Gaussian motion model on translation and rotation; repeatedly sampling it along a path produces a sample cloud that grows and bends with the accumulated heading uncertainty. Sampling and weighting are easy, but the proposal ignores z_t. When the measurement likelihood is much sharper than the motion model, most particles fall where $p(z_t \mid x_t)$ is tiny, giving many low-weight samples.

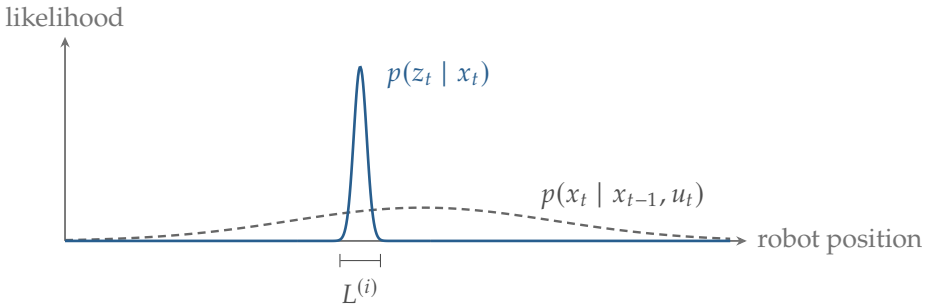


Figure 29: A peaked measurement likelihood (blue) and a broad motion model (dashed). Only particles proposed inside the narrow region $L^{(i)}$ receive appreciable weight. Curves are illustrative.

5.7 Resampling

The second remedy eliminates particles with small normalized weight and concentrates on those with large weight. The idea is to resample only when the particles are ineffective. With normalized weights $\tilde{w}_t^{(i)}$, the EFFECTIVE SAMPLE SIZE

$$\hat{N}_{\text{eff}} = \frac{1}{\sum_{i=1}^{N} \left(\tilde{w}_t^{(i)}\right)^2}$$

is inversely related to the variance of the weights. If all $\tilde{w}_t^{(i)} = 1/N$, then $\hat{N}_{\text{eff}} = N$; if all but one weight is zero, then $\hat{N}_{\text{eff}} = 1$.

Algorithm 5.3: Sequential importance resampling (SIR)

for $k = 0, 1, 2, \ldots$ **do**
  for each $i \in \{1, \ldots, n\}$ **do**
    Sample $x_k^{(i)} \sim q\left(x_k \mid x_{0:k-1}^{(i)}, z_{1:k}\right)$ and set $x_{0:k}^{(i)} = \left(x_{0:k-1}^{(i)}, x_k^{(i)}\right)$
    $w_k^{(i)} \leftarrow w_{k-1}^{(i)} \dfrac{p(z_k \mid x_k^{(i)})\, p(x_k^{(i)} \mid x_{k-1}^{(i)})}{q(x_k^{(i)} \mid x_{0:k-1}^{(i)}, z_{1:k})}$
  for each $i \in \{1, \ldots, n\}$ **do**
    $\tilde{w}_k^{(i)} \leftarrow w_k^{(i)} / \sum_{j=1}^{n} w_k^{(j)}$
  if $\hat{N}_{\text{eff}} \le N_{\text{thresh}}$ **then**
    Draw n particles with replacement with probabilities $\tilde{w}_k^{(i)}$ and reset all weights to $1/n$

5.7.1 Robot localization

In the hallway example with the Markov-prior proposal, the particles start uniformly spread. A door detection reweights them by $w_t^{(i)} = w_{t-1}^{(i)}\, p(z_t \mid x_t = x_t^{(i)})$, so the particles near the three doors gain weight. Resampling duplicates those particles; motion then shifts each one by a sample of the motion model, spreading the clusters. A second door detection makes only the cluster consistent with both observations heavy, and after resampling most particles sit there.

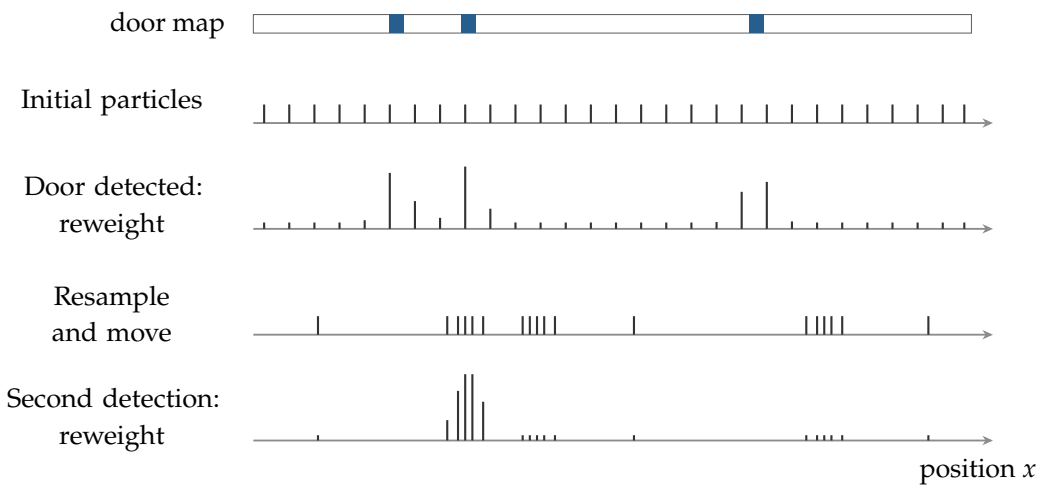


Figure 30: Schematic particle filter in the hallway. Bar positions are particles and bar heights are their weights. Particles near doors gain weight; after resampling and motion, only the cluster that moved from the first door to the second is supported by the second detection. Particle positions are illustrative.

The same procedure localizes a robot in a building map: the initial particles fill the free space, after some motion and laser measurements they form a few clusters at plausible poses, and eventually a single tight cluster remains at the true pose.

5.7.2 Issues with resampling

Consider a SIR filter for a system with deterministic motion and no sensors. All particles keep equal weight, yet resampling still removes some particles at random. As time increases, all particles become identical with high probability, which falsely suggests that the filter has converged to a single deterministic estimate. In general, resampling loses diversity, and it is not a parallel operation, since every draw depends on all the weights.

Particle depletion means that no particles are near the true state. It can occur when $p(z_t \mid x_t)$ and $p(x_{0:t} \mid z_{0:t-1})$ have little overlap. More fundamentally, the randomness of resampling means that, with nonzero probability, all particles near the true state are eliminated; with enough resampling, this will happen. Two remedies are common.

- *Roughening:* add a small amount of noise, such as Gaussian noise, to each particle after resampling.
- *Random particles:* add a small number of randomly generated particles when resampling. This reduces the likelihood of depletion, helps when the initial particles include no samples near the true state, and is simple. The posterior is then incorrect, however, and stays incorrect even with infinitely many particles.

Source. Matthew R. Walter, TTIC 31170, Lecture 5: Nonparametric Filters, October 12, 2026.

Lecture 6: Simultaneous Localization and Mapping

6.1 What the robot knows, and what it must estimate

Suppose a robot drives through a building that it has never visited. Its wheels provide an estimate of how far it has moved. A range sensor reports distances to nearby objects. Neither source is exact. Wheels can slip, a heading estimate can drift, and the sensor readings contain noise. The task is to use these imperfect measurements to estimate both the robot's location and a map of the building.

The robot's pose describes its position and orientation. For a robot moving on a plane, a common pose vector is $x_t = (p_{x,t}, p_{y,t}, \theta_t)^\top$: two coordinates and a heading at time t. A landmark is a feature that can be recognized and measured again, such as a corner or a pole. A feature map stores the locations of these landmarks in a common coordinate system.

Three closely related tasks should be distinguished.

- *Localization:* the map is given, and the robot estimates its pose. A measured distance can be compared with distances predicted from the known map.
- *Mapping:* the robot's path is given, and the robot estimates the map. A local sensor reading can be placed in global coordinates using the known pose.
- *Simultaneous localization and mapping (SLAM):* both the path and map are unknown. The robot must estimate them using the same observations.

The difficulty is their dependence. To place a corner in a map, the robot needs to know where it was when it saw the corner. To use that corner to locate itself later, it needs to know where the corner is. A probabilistic solution keeps their uncertainty together rather than treating one estimate as exact while estimating the other.

6.1.1 Why motion estimates alone are insufficient

Odometry estimates motion from the robot's movement, for example from wheel rotations. Integrating these motions produces a path estimate. Small errors accumulate because every new position is computed from an already uncertain position. Heading error is especially troublesome: driving forward with a slightly wrong heading shifts all later observations sideways. Walls seen at different times may then be placed at inconsistent locations in the map.

A loop closure occurs when the robot recognizes a previously visited place. This supplies a new constraint between the current part of the path and an earlier part. If the motion estimate says the robot has not quite returned to its starting point but the observations identify the same place, the

two sources disagree. SLAM uses their uncertainties to decide how to revise the path and map. It does not simply declare one source correct.

6.1.2 Notation and the meaning of a joint belief

We will use the following notation throughout this lecture.

Symbol	Meaning
x_t	Robot pose at time t.
$m = (m_1, \ldots, m_n)$	Static map containing n landmarks. Each m_j is a location vector.
u_t	Control or motion input for the transition from $t-1$ to t.
z_t	Sensor observation at time t; it may contain several feature measurements.
$u_{1:t}, z_{1:t}$	All controls and observations through time t.
$x_{0:t}$	The complete sequence of poses from the initial time through t.

The map is assumed static: its unknown value is the same at every time. This does not mean that its estimate stays fixed. New data can change what the robot believes about a stationary landmark.

A JOINT BELIEF assigns probability to combinations of pose and map. For example, a robot might be at position 0 with a landmark at 3, or at position 1 with the landmark at 4. Both combinations predict a relative distance of 3. A sensor that measures only that distance may distinguish these combinations from other combinations without distinguishing them from each other.

This example shows why keeping only separate pose and landmark distributions loses information. Knowing that the robot is likely near either 0 or 1 and that the landmark is near either 3 or 4 does not record which pairs go together. The joint distribution records that relationship.

6.1.3 Choosing what the map represents

A map need not be a list of landmarks. An OCCUPANCY GRID divides space into cells and records whether each is occupied or free, with uncertainty. A TOPOLOGICAL MAP records places and connections, such as rooms joined by doorways. A FEATURE MAP records geometric objects and their locations. The representation determines what a measurement refers to and what the robot can later ask of its map. We use feature maps in the EKF calculation below.

Different SLAM methods also use different ways to represent uncertainty and constraints. Scan matching aligns overlapping sensor scans; EKF and information filters maintain Gaussian approximations; particle-based approaches include FastSLAM; pose-graph methods optimize constraints across a trajectory. The derivations here concern Gaussian feature-based SLAM and the graph formulation that follows from it.

6.2 Online SLAM and full SLAM

Online SLAM estimates the current pose and the map:

$$b_t(x_t, m) = p(x_t, m \mid z_{1:t}, u_{1:t}).$$

Here b_t is a name for the belief, not an additional random variable. Full SLAM keeps the whole trajectory:

$$p(x_{0:t}, m \mid z_{1:t}, u_{1:t}).$$

A later observation can then revise a past pose as well as the present pose. Online SLAM removes old poses by integration, a process called marginalization. Its belief is the marginal of the full posterior:

$$b_t(x_t, m) = \int p(x_{0:t}, m \mid z_{1:t}, u_{1:t})\, dx_0 \cdots dx_{t-1}.$$

Removing a variable from the stored state does not remove the information it contributed. As we will see in the next lecture, its effect remains as dependencies among the variables that are retained.

6.2.1 The assumptions behind the filter

The recursive calculation uses two model assumptions.

1. *Motion is Markov.* Given the previous pose and the current control, the new pose does not need the earlier history. We use $p(x_t \mid x_{t-1}, u_t)$. In this model, motion does not depend directly on the unknown map.
2. *An observation is local in time.* Given the current pose and map, the current observation does not need earlier poses, controls, or observations. We use $p(z_t \mid x_t, m)$.

The observation model must include the feature that was observed. For now, assume its identity is known. Deciding that identity is the data-association problem discussed below.

The graphical model makes the assumptions visible. Pose-to-pose arrows describe motion. An observation has the current pose and the map as parents because both determine what the sensor should see. Shaded nodes are known observations.

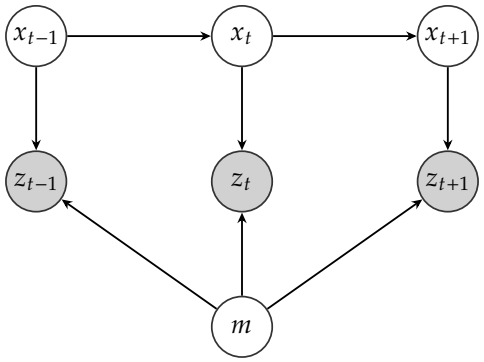


Figure 31: The static map is shared across time. Controls are omitted from the drawing; u_t is an input to the transition into x_t.

6.2.2 Deriving the prediction

Assume that $b_{t-1}(x_{t-1}, m)$ has already incorporated the previous observations. Before using z_t, define the predicted belief

$$\bar{b}_t(x_t, m) = p(x_t, m \mid z_{1:t-1}, u_{1:t}).$$

The bar means that the latest motion has been incorporated but the latest observation has not. Introduce the previous pose and integrate it out:

$$\begin{aligned}
\bar{b}_t(x_t, m) &= \int p(x_t, x_{t-1}, m \mid z_{1:t-1}, u_{1:t})\, dx_{t-1} \\
&= \int p(x_t \mid x_{t-1}, m, z_{1:t-1}, u_{1:t})\, p(x_{t-1}, m \mid z_{1:t-1}, u_{1:t})\, dx_{t-1} \\
&= \int p(x_t \mid x_{t-1}, u_t)\, b_{t-1}(x_{t-1}, m)\, dx_{t-1}.
\end{aligned}$$

The second line is the probability product rule. The last line uses the motion assumption and treats controls as given inputs. The integral averages over uncertainty in the old pose. For each possible old pose, the motion model supplies a distribution of new poses. We add these possibilities with the weights supplied by the old belief.

We integrate over the old pose, not over the map. The map remains part of the new state, and its correlations with the pose remain relevant.

6.2.3 Deriving the measurement correction

Bayes' rule gives the corrected belief:

$$\begin{aligned} b_t(x_t, m) &= \frac{p(z_t \mid x_t, m, z_{1:t-1}, u_{1:t})\, p(x_t, m \mid z_{1:t-1}, u_{1:t})}{p(z_t \mid z_{1:t-1}, u_{1:t})} \\ &= \frac{p(z_t \mid x_t, m)\, \bar{b}_t(x_t, m)}{Z_t}, \end{aligned}$$

where the normalizing constant is

$$Z_t = \int\int p(z_t \mid x_t, m)\, \bar{b}_t(x_t, m)\, dx_t\, dm.$$

The likelihood $p(z_t \mid x_t, m)$ asks how plausible the actual observation would be for a proposed pose-map pair. Multiplying by it increases the relative weight of pairs that fit the reading. Dividing by Z_t makes the posterior integrate to one. These equations define the Bayesian calculation; an implementation must still choose how to represent its distribution.

6.3 A Gaussian representation of pose and landmarks

EKF SLAM uses a feature map and approximates the joint belief by a multivariate Gaussian. Let the pose have d coordinates and each landmark have p coordinates. Stack the unknowns in the column vector

$$\xi_t = \begin{bmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{bmatrix} \in \mathbb{R}^D, \qquad D = d + np.$$

The approximation is $\xi_t \sim \mathcal{N}(\mu_t, \Sigma_t)$. The mean vector μ_t contains the pose estimate followed by the landmark estimates. The $D \times D$ covariance matrix Σ_t describes uncertainty in all these entries together.

For readability, group all landmarks into a single map block:

$$\mu_t = \begin{bmatrix} \hat{x}_t \\ \hat{m}_t \end{bmatrix}, \qquad \Sigma_t = \begin{bmatrix} P_t & C_t \\ C_t^\top & M_t \end{bmatrix}.$$

Here $P_t = \mathrm{Cov}(x_t, x_t)$ is pose covariance, $M_t = \mathrm{Cov}(m, m)$ is map covariance, and $C_t = \mathrm{Cov}(x_t, m)$ is pose-map cross-covariance. Covariance is an expectation of error products: $\mathrm{Cov}(a, b) = \mathbb{E}[(a - \mathbb{E}[a])(b - \mathbb{E}[b])^\top]$.

A diagonal variance measures uncertainty in one coordinate. A cross-covariance describes how two estimation errors vary together. In one dimension, positive cross-covariance means that an

error shifting the pose to the right tends to come with an error shifting the landmark to the right. This is natural for a relative-distance sensor: shifting both by the same amount does not change the predicted reading.

6.4 Deriving the EKF prediction

Use the motion model

$$x_t = f_t(x_{t-1}, u_t) + w_t, \qquad w_t \sim \mathcal{N}(0, R_t).$$

The function f_t predicts pose from previous pose and control; R_t is the $d \times d$ motion-noise covariance. Assume the new noise is independent of the previous state. Because the map is static, the augmented motion function is

$$\phi_t(\xi_{t-1}, u_t) = \begin{bmatrix} f_t(x_{t-1}, u_t) \\ m \end{bmatrix}.$$

The model can be nonlinear, so a Gaussian input need not give a Gaussian output. The extended Kalman filter uses a first-order Taylor approximation around the previous mean.

6.4.1 Mean and error propagation

Let F_t denote the pose Jacobian, the matrix of derivatives of f_t with respect to pose, evaluated at $\hat{x}_{t-1}$. With the control held fixed,

$$f_t(x_{t-1}, u_t) \approx f_t(\hat{x}_{t-1}, u_t) + F_t(x_{t-1} - \hat{x}_{t-1}).$$

Since the pose error and noise have zero mean, the predicted mean is

$$\bar{\mu}_t = \begin{bmatrix} f_t(\hat{x}_{t-1}, u_t) \\ \hat{m}_{t-1} \end{bmatrix}.$$

The landmark means stay unchanged. The full Jacobian and full noise covariance are

$$\mathcal{F}_t = \begin{bmatrix} F_t & 0 \\ 0 & I \end{bmatrix}, \qquad \mathcal{R}_t = \begin{bmatrix} R_t & 0 \\ 0 & 0 \end{bmatrix},$$

where I acts on the map coordinates. If $e_{t-1} = \xi_{t-1} - \mu_{t-1}$, the predicted error is approximately $\mathcal{F}_t e_{t-1} + (w_t, 0)^\top$. Taking its covariance gives

$$\bar{\Sigma}_t = \mathcal{F}_t \Sigma_{t-1} \mathcal{F}_t^\top + \mathcal{R}_t.$$

The mixed noise-state terms vanish by independence. Multiplying the blocks shows exactly what changes:

$$\boxed{\bar{\Sigma}_t = \begin{bmatrix} F_t P_{t-1} F_t^\top + R_t & F_t C_{t-1} \\ C_{t-1}^\top F_t^\top & M_{t-1} \end{bmatrix}.}$$

The map covariance is unchanged during prediction. The pose covariance changes, and its cross-covariance with the map is transformed by F_t. Treating the map as static therefore does not justify setting pose-map correlations to zero.

6.5 Deriving the measurement update

Suppose an observation of known landmarks is described by

$$z_t = h_t(\xi_t) + v_t, \qquad v_t \sim \mathcal{N}(0, Q_t).$$

The measurement has k entries, Q_t is $k \times k$, and its new noise is independent of the predicted state. The function h_t converts a proposed pose and map into a predicted sensor reading.

Evaluate its $k \times D$ Jacobian H_t at $\bar{\mu}_t$. Linearization gives

$$h_t(\xi_t) \approx h_t(\bar{\mu}_t) + H_t(\xi_t - \bar{\mu}_t).$$

Define the INNOVATION $r_t = z_t - h_t(\bar{\mu}_t)$. Substitution yields

$$r_t \approx H_t e_t + v_t, \qquad e_t = \xi_t - \bar{\mu}_t.$$

The innovation is the part of the reading not predicted by the current estimate. Its covariance and its cross-covariance with state error are

$$\begin{aligned} S_t &= \mathbb{E}[(H_t e_t + v_t)(H_t e_t + v_t)^\top] = H_t \bar{\Sigma}_t H_t^\top + Q_t, \\ \mathrm{Cov}(e_t, r_t) &= \mathbb{E}[e_t (H_t e_t + v_t)^\top] = \bar{\Sigma}_t H_t^\top. \end{aligned}$$

Again, independence makes the mixed terms zero. S_t measures how uncertain the predicted observation is, including both state uncertainty and sensor noise.

6.5.1 Where the Kalman gain comes from

Seek a corrected estimate of the form $\mu_t = \bar{\mu}_t + K_t r_t$. The matrix K_t translates an error in measurement coordinates into a correction in state coordinates. After correction, the state error is $e_t - K_t r_t$. At the best linear correction, this remaining error is uncorrelated with the innovation: otherwise the

innovation would still contain a linearly usable prediction of the error. Thus

$$\begin{aligned} 0 &= \operatorname{Cov}(e_t - K_t r_t, r_t) \\ &= \bar{\Sigma}_t H_t^\top - K_t S_t, \end{aligned}$$

which gives

$$K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}, \qquad \boxed{\mu_t = \bar{\mu}_t + K_t \big[z_t - h_t(\bar{\mu}_t) \big].}$$

For the linearized Gaussian model this is also the conditional mean. In practice, solve a linear system with S_t rather than explicitly computing its inverse.

This uncorrelated-error condition also proves that the gain is optimal among linear corrections. Use another gain $K_t + \Delta$. Its error is $(e_t - K_t r_t) - \Delta r_t$. The cross terms in its covariance vanish because $e_t - K_t r_t$ is uncorrelated with r_t. Its covariance is therefore the covariance for K_t plus $\Delta S_t \Delta^\top$, a positive semidefinite matrix. No other linear gain can reduce the variance in any direction below this result.

The new error is $(I - K_t H_t) e_t - K_t v_t$. Taking its covariance gives

$$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t (I - K_t H_t)^\top + K_t Q_t K_t^\top.$$

This is the Joseph form. Expanding it and using $K_t S_t = \bar{\Sigma}_t H_t^\top$ gives the equivalent expressions

$$\Sigma_t = \bar{\Sigma}_t - K_t S_t K_t^\top = (I - K_t H_t) \bar{\Sigma}_t.$$

The Joseph form is often preferable in floating-point calculations because the two terms visibly have the form of covariance contributions. The EKF still makes an approximation: these identities are exact for the linearized model, not for an arbitrary nonlinear posterior.

6.5.2 Why one observation can change the whole map

A measurement of landmark j depends on the robot and that landmark. The columns of H_t for other landmarks are zero. Nevertheless, the rows of $\bar{\Sigma}_t H_t^\top$ for another landmark ℓ can be nonzero because of its cross-covariances with the robot or landmark j. Therefore the corresponding rows of K_t can be nonzero. That unobserved landmark is corrected through shared uncertainty.

This is how a loop closure can correct more than the local region in view. It also explains why keeping only the diagonal of the covariance changes the method: doing so erases the relationships that carry this correction.

Consider a sequence involving landmarks A, B, and C. The robot first observes A near its starting position. After moving, its pose is less certain, so newly located B and C inherit uncertainty from that pose. It then returns and recognizes A. Correcting the robot against A can also improve B and C even if they are not currently visible. Later reobserving B can reduce uncertainty about C

because they were placed using a shared uncertain pose. The covariance blocks record the common errors that make these changes possible.

6.6 Growing the map without losing shared uncertainty

Data association determines which mapped feature, if any, generated each observation. The distinction changes the operation we perform.

- *Known feature:* compare the reading with its predicted reading and apply a measurement correction.
- *New feature:* create a new state variable using an inverse observation model, and add its uncertainty and cross-covariances to the state.

An incorrect match adds a false geometric relationship. Large pose uncertainty makes this harder because several landmarks may produce plausible readings. The filter equations assume the selected match is correct; they do not by themselves solve recognition.

6.6.1 From a local reading to a global feature

An inverse observation model g converts the pose and a new measurement into a landmark location. For example, with planar pose (p_x, p_y, θ) and a measured range r and bearing β,

$$g(x, z) = \begin{bmatrix} p_x + r\cos(\theta + \beta) \\ p_y + r\sin(\theta + \beta) \end{bmatrix}, \qquad z = \begin{bmatrix} r \\ \beta \end{bmatrix}.$$

The sensor's bearing is relative to the robot heading. Adding θ converts it to a global direction; multiplying by r gives a displacement; adding the robot position gives the landmark position. Angles and angle differences must use a consistent convention.

Let $a = \theta + \beta$. The derivatives with respect to pose and measurement are

$$G_x = \begin{bmatrix} 1 & 0 & -r\sin a \\ 0 & 1 & r\cos a \end{bmatrix}, \qquad G_z = \begin{bmatrix} \cos a & -r\sin a \\ \sin a & r\cos a \end{bmatrix}.$$

These derivatives also explain a geometric effect: heading uncertainty causes greater location uncertainty for distant landmarks because the heading column contains the range r.

6.6.2 Deriving the augmented covariance

For this step, write the existing state as $\xi \sim \mathcal{N}(\mu, \Sigma)$, after any corrections using known features. Let the new reading have independent noise covariance Q. The new mean is $\hat{m}_{\text{new}} = g(\hat{x}, z)$. Define $J = [G_x \;\; 0]$, with zero columns for the existing map. Linear uncertainty propagation gives the new feature error approximately as

$$e_{\text{new}} \approx Je + G_z\epsilon,$$

where $e = \xi - \mu$ and ϵ represents the measurement error. A possible sign change in the definition of ϵ does not affect its covariance. Independence gives

$$\begin{aligned} \text{Cov}(e, e_{\text{new}}) &= \mathbb{E}[e(Je + G_z\epsilon)^\top] = \Sigma J^\top, \\ \text{Cov}(e_{\text{new}}, e_{\text{new}}) &= J\Sigma J^\top + G_z Q G_z^\top. \end{aligned}$$

Therefore the expanded state has parameters

$$\boxed{\mu^+ = \begin{bmatrix} \mu \\ g(\hat{x}, z) \end{bmatrix}, \qquad \Sigma^+ = \begin{bmatrix} \Sigma & \Sigma J^\top \\ J\Sigma & J\Sigma J^\top + G_z Q G_z^\top \end{bmatrix}.}$$

The top-left block preserves the old belief. The bottom-right block includes both robot uncertainty and sensor uncertainty. The off-diagonal blocks record the fact that the new feature was located using the uncertain robot pose. Initializing only a landmark variance while setting these blocks to zero would discard information.

The reading used to initialize the new variable must not then be treated as a second independent observation. It has already contributed to the augmented distribution. Reusing it as new evidence would count the same information twice.

6.6.3 The order of operations

Start with a prior for the robot pose and whatever map is already available. Each iteration predicts using the new control, associates the new readings, corrects with known features, and initializes genuinely new features. If the scan contains no known features, the correction step is skipped. If it contains no new features, the state size stays unchanged.

Algorithm 6.1: One EKF SLAM iteration

Input: Previous joint mean and covariance, control u_t, observations z_t
Output: Updated joint mean and covariance, with any new landmarks appended

```
Evaluate the motion model and its Jacobian at the previous pose mean
Predict the joint mean and covariance using the static-map motion model
Associate observations with known features or classify them as new
if known-feature observations are available then
    Evaluate their predicted readings and observation Jacobian
    Compute the innovation, S_t, and Kalman gain K_t
    Correct the joint mean and covariance
if new-feature observations are available then
    Evaluate the inverse observation model and its Jacobians
    Append the new means, variances, and cross-covariances
return Updated joint mean and covariance
```

When initializing several features from the same pose, they also share pose uncertainty with one another. Apply the augmentation to the stacked new features or preserve the resulting cross-covariances in sequential initialization. Treating all new features as independent would lose the relationship between them.

6.7 A complete one-dimensional SLAM calculation

The following invented scalar example follows a feature through initialization, motion, and a later observation. It uses linear models so that the Gaussian calculations are exact. Positions are measured along one line.

6.7.1 Step 1: observe a new landmark

Initially, the robot has $x_0 \sim \mathcal{N}(2, 4)$. A sensor measures landmark position relative to robot position: $z_0 = m - x_0 + v_0$, where v_0 has variance 1. The reading is $z_0 = 3$, so the estimated landmark location is $2 + 3 = 5$. Both inverse-model derivatives are 1. Hence

$$\mu_0 = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, \qquad \Sigma_0 = \begin{bmatrix} 4 & 4 \\ 4 & 5 \end{bmatrix}.$$

The landmark variance is $4 + 1 = 5$. Its covariance with the robot is 4, because both estimates share the uncertain global robot position. Their relative-position variance is $4 + 5 - 2(4) = 1$, exactly the sensor variance.

6.7.2 Step 2: move the robot

The next motion is $x_1 = x_0 + 1 + w_1$, with independent motion-noise variance 1. Since the derivative with respect to x_0 is 1, prediction gives

$$\bar{\mu}_1 = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \qquad \bar{\Sigma}_1 = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}.$$

Only the robot variance increases. The map stays still, and the old cross-covariance remains. The predicted relative distance is $5 - 3 = 2$; its variance is now $5 + 5 - 2(4) = 2$.

6.7.3 Step 3: observe the same landmark again

A second sensor reading is $z_1 = 3/2$, with noise variance 1. This is a known landmark, so it supplies a correction rather than another map variable. The observation Jacobian is $H = [-1 \;\; 1]$. The innovation and its variance are

$$r = \frac{3}{2} - 2 = -\frac{1}{2}, \qquad S = H\bar{\Sigma}_1 H^\top + 1 = 2 + 1 = 3.$$

The gain uses both diagonal and off-diagonal covariance entries:

$$\bar{\Sigma}_1 H^\top = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \qquad K = \begin{bmatrix} -1/3 \\ 1/3 \end{bmatrix}.$$

Applying the innovation gives

$$\mu_1 = \begin{bmatrix} 3 \\ 5 \end{bmatrix} + \begin{bmatrix} -1/3 \\ 1/3 \end{bmatrix} \left(-\frac{1}{2} \right) = \begin{bmatrix} 19/6 \\ 29/6 \end{bmatrix}.$$

The robot moves right by $1/6$ and the landmark moves left by $1/6$ in the estimate. The estimated distance becomes $5/3$, between the predicted distance 2 and measured distance $3/2$.

For the covariance, use $\Sigma_1 = \bar{\Sigma}_1 - KSK^\top$:

$$KSK^\top = \frac{1}{3} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \qquad \Sigma_1 = \begin{bmatrix} 14/3 & 13/3 \\ 13/3 & 14/3 \end{bmatrix}.$$

The relative-distance variance falls from 2 to $14/3 + 14/3 - 2(13/3) = 2/3$. Yet the individual position variances remain large. The sensor has made the geometry between robot and landmark more certain without directly revealing where the whole configuration lies globally.

6.8 What limits EKF SLAM, and why pose graphs help

With fixed pose and feature dimensions, the joint state length grows linearly with the number of landmarks. A dense covariance, with many nonzero entries, has quadratically many entries. Even a fixed-size observation can require a quadratic covariance correction because correlations connect the observed part to the rest of the map. This is why a small sensor reading does not necessarily imply a cheap update.

There are also statistical limits. A first-order Taylor approximation can be poor when pose or heading uncertainty is large. Incorrect data association can impose a false loop closure. Keeping correlations is necessary for the intended EKF calculation, but it does not remove these modeling and recognition problems.

Large-scale systems often combine three tasks: build local metric maps, recognize previously visited places, and optimize the global arrangement when loop closures are found. A POSE GRAPH records poses as nodes and relative constraints as edges; a graph can also include explicit landmark nodes. Motion supplies constraints between consecutive poses. Reobservations supply constraints between poses and landmarks or between revisited places.

Graph optimization looks for a set of positions that jointly fits these noisy constraints. It allows a loop-closure discrepancy to be distributed across several estimates instead of attaching all of it to the current pose. The next lecture derives this calculation as a weighted least-squares problem and explains why retaining old poses can preserve useful sparse structure.

Source. Matthew R. Walter, TTIC 31170, Lecture 6: Simultaneous Localization and Mapping, October 14, 2026.

Lecture 7: Simultaneous Localization and Mapping (Continued)

7.1 Why change the Gaussian representation?

The previous lecture represented the joint pose-map belief by a mean vector and a covariance matrix. The covariance records how estimation errors vary together. This is useful, but correlations spread through the map, so the matrix generally becomes dense. A dense matrix has many nonzero entries; a sparse matrix has many entries equal to zero. Storing and updating all the covariance entries becomes expensive as the map grows.

This lecture studies two responses. First, we represent the same Gaussian using an information matrix and examine which operations become easier. Second, we keep old poses and express full SLAM as an optimization problem. The important question is what happens to dependencies when we add measurements or remove variables. A sparse matrix is useful only if the required operations preserve its sparsity, or if the approximation used to maintain sparsity is understood.

Throughout, ξ is a column vector containing the current robot pose and map variables. Its dimension is D. When a time index is not needed, we omit it. The robot pose is x, the complete feature map is m, and the previous lecture's covariance representation is $\xi \sim \mathcal{N}(\mu, \Sigma)$.

7.2 Deriving Gaussian information form

Assume Σ is positive definite, so that it has an inverse. The Gaussian density is

$$p(\xi) = \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\xi-\mu)^\top \Sigma^{-1}(\xi-\mu)\right].$$

Here $|\Sigma|$ denotes the determinant. For calculations involving the unknown ξ, the coefficient outside the exponential is a constant. We can absorb such constants into a normalizer and write a proportionality.

Expand the quadratic inside the exponent:

$$\begin{aligned}(\xi-\mu)^\top \Sigma^{-1}(\xi-\mu) &= \xi^\top\Sigma^{-1}\xi - \xi^\top\Sigma^{-1}\mu - \mu^\top\Sigma^{-1}\xi + \mu^\top\Sigma^{-1}\mu \\ &= \xi^\top\Sigma^{-1}\xi - 2\xi^\top\Sigma^{-1}\mu + \mu^\top\Sigma^{-1}\mu.\end{aligned}$$

The two middle scalars are equal because Σ^{-1} is symmetric. The last term does not contain ξ, so it also becomes part of the normalizing constant. This leaves

$$p(\xi) \propto \exp\left[-\frac{1}{2}\xi^\top\Lambda\xi + \eta^\top\xi\right], \qquad \boxed{\Lambda = \Sigma^{-1}, \quad \eta = \Lambda\mu.}$$

Λ is the INFORMATION MATRIX, also called the precision matrix. η is the INFORMATION VECTOR. They are alternative parameters of the same Gaussian, not additional observations.

7.2.1 Recovering the mean

The mean is the solution of $\Lambda\mu = \eta$. One way to see this is to differentiate the log density:

$$\nabla_\xi \log p(\xi) = -\Lambda\xi + \eta.$$

Its maximum is at $\xi = \Lambda^{-1}\eta$. A Gaussian is symmetric around its mean, so this maximum is its mean. In a computation, solve the system rather than first forming the inverse. Recovering the entire covariance requires $\Sigma = \Lambda^{-1}$ and can be much more expensive than storing a sparse Λ.

Example: In one dimension, let $x \sim \mathcal{N}(2, 4)$. Its precision is $\Lambda = 1/4$ and its information vector is $\eta = (1/4) \cdot 2 = 1/2$. The equation $(1/4)\mu = 1/2$ gives back $\mu = 2$. A smaller variance gives a larger precision. In several dimensions, off-diagonal entries also describe how variables are related, so precision is more than a separate confidence number for each coordinate.

7.3 What the information matrix says about a graph

For scalar coordinates, expand the exponent once more:

$$-\frac{1}{2}\xi^\top\Lambda\xi + \eta^\top\xi = \sum_i \left(-\frac{1}{2}\Lambda_{ii}\xi_i^2 + \eta_i\xi_i\right) - \sum_{i<j}\Lambda_{ij}\xi_i\xi_j.$$

Each term in the first sum involves one variable. Each term in the second involves a pair. Exponentiating turns the sum into a product of single-variable and pairwise factors. This is an undirected Gaussian graphical model: a nonzero Λ_{ij} supplies an edge between i and j.

For a nonsingular Gaussian, $\Lambda_{ij} = 0$ means that ξ_i and ξ_j are conditionally independent given every other coordinate. To understand why, hold those other coordinates fixed. The remaining exponent contains separate quadratic and linear terms in ξ_i and ξ_j. If the cross term is absent, their conditional density factors into a function of ξ_i times a function of ξ_j. For vector-valued poses or features, replace an entry by the corresponding matrix block.

Conditional independence is different from zero marginal correlation. Two landmarks may be independent once the robot pose is known, yet correlated when that pose is uncertain. The covariance matrix describes the latter relationship. Its inverse can therefore be sparse even when the covariance is dense.

7.3.1 A numerical example of the distinction

Consider zero-mean scalar variables (a_1, a_2, b) with information matrix

$$\Lambda = \begin{bmatrix} 2 & 0 & -1 \\ 0 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}.$$

There is no direct edge between a_1 and a_2, because $\Lambda_{12} = 0$. They are conditionally independent given b. Nevertheless, the covariance is

$$\Sigma = \Lambda^{-1} = \begin{bmatrix} 3/4 & 1/4 & 1/2 \\ 1/4 & 3/4 & 1/2 \\ 1/2 & 1/2 & 1 \end{bmatrix}.$$

The covariance between a_1 and a_2 is $1/4$, not zero. If the shared variable b is uncertain, an observation of one can tell us about the other through b.



Before removing b After marginalizing b

Figure 32: Removing an uncertain shared variable creates a direct dependence between its neighbors. The dashed edge is a fill edge, not an original measurement.

7.3.2 Active and passive landmarks

In an information-filter map, a landmark is ACTIVE when its information-matrix block with the robot pose is nonzero. It is PASSIVE when that block is zero. A passive landmark can still be correlated with the robot through other variables. These terms refer to direct conditional dependence, not to whether a feature was ever observed or whether it has zero uncertainty.

The active set matters because removing a pose creates new links among its neighbors. If the pose has many active landmarks, eliminating it can affect many landmark pairs.

7.4 Deriving an information-form observation update

Let $(\bar{\eta}, \bar{\Lambda})$ describe the predicted Gaussian before a measurement. A bar again means that the observation has not yet been incorporated. Suppose

$$z = h(\xi) + v, \qquad v \sim \mathcal{N}(0, Q),$$

with independent noise. As in the EKF, linearize about a state estimate $\bar{\mu}$. Define

$$H = \left.\frac{\partial h}{\partial \xi}\right|_{\bar{\mu}}, \qquad c = h(\bar{\mu}) - H\bar{\mu}.$$

Then $h(\xi) \approx H\xi + c$. The offset c is needed because the tangent line or plane does not generally pass through the origin. Write $y = z - c = z - h(\bar{\mu}) + H\bar{\mu}$. The linearized observation is $y \approx H\xi + v$.

7.4.1 Expanding the likelihood

The likelihood is proportional to

$$\exp\left[-\frac{1}{2}(y - H\xi)^\top Q^{-1}(y - H\xi)\right].$$

Expand its quadratic:

$$(y - H\xi)^\top Q^{-1}(y - H\xi) = y^\top Q^{-1} y - 2\xi^\top H^\top Q^{-1} y + \xi^\top H^\top Q^{-1} H\xi.$$

The first term is constant in ξ. Therefore the likelihood contributes $H^\top Q^{-1} H$ to the quadratic coefficient and $H^\top Q^{-1} y$ to the linear coefficient. Bayes' rule multiplies the prior by this likelihood, so their exponents add:

$$\boxed{\Lambda^+ = \bar{\Lambda} + H^\top Q^{-1} H, \qquad \eta^+ = \bar{\eta} + H^\top Q^{-1}\big[z - h(\bar{\mu}) + H\bar{\mu}\big].}$$

The superscript + denotes the result after the observation. This is the extended information filter (EIF) correction. For a linear observation, the update is exact for the Gaussian model. For a nonlinear observation, it depends on the chosen linearization point.

7.4.2 A complete update with two scalar variables

Take $\xi = (x, m)^\top$ with independent prior variables of mean 0 and variance 1. Thus $\bar{\Lambda} = I$ and $\bar{\eta} = 0$. A sensor measures $z = m - x + v$, with noise variance 1, and reports $z = 2$. Here $H = [-1 \;\; 1]$, and

there is no offset. Consequently

$$H^\top H = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \qquad H^\top z = \begin{bmatrix} -2 \\ 2 \end{bmatrix}.$$

After adding the measurement,

$$\Lambda^+ = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad \eta^+ = \begin{bmatrix} -2 \\ 2 \end{bmatrix}.$$

The corrected mean solves $2\mu_x - \mu_m = -2$ and $-\mu_x + 2\mu_m = 2$. Hence $\mu_x = -2/3$ and $\mu_m = 2/3$. The posterior covariance is

$$(\Lambda^+)^{-1} = \frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The reading says the landmark lies to the robot's right. Both estimates move to make that possible, and their errors become correlated. The new off-diagonal precision entry is the graph edge supplied by the relative measurement.

7.4.3 What is cheap, and what still costs work?

If the reading involves the robot and one landmark, H has nonzero columns only for those variables. The added matrix touches only their blocks. With a fixed number of observed features and known local mean estimates, adding the observation can take an amount of work independent of the total map size.

There are two qualifications. A batch involving more features can touch more blocks. Also, a nonlinear observation needs an appropriate mean around which to linearize. Recovering that mean from the information parameters requires solving a system; a cheap local addition does not imply that every part of the filter is cheap.

7.5 Conditioning and marginalization are different operations

Partition the state as $\xi = (a, b)$, where a contains variables we want to retain. Partition its parameters in the same order:

$$\Lambda = \begin{bmatrix} A & B \\ B^\top & C \end{bmatrix}, \qquad \eta = \begin{bmatrix} u \\ v \end{bmatrix}.$$

Here A acts on a, C acts on b, and B links the two groups. The letters in this subsection are block names, not motion or observation Jacobians.

- *Conditioning on $b = b_0$* means the value of b has been learned exactly. Substitute that known value into the density.
- *Marginalizing b* means its value remains unknown, but we no longer store it. Integrate over all possible values, weighted by their probabilities.

These operations answer different questions and usually give different uncertainty.

7.5.1 Conditioning in information form

With $b = b_0$, the terms involving a in the exponent are

$$-\frac{1}{2}a^\top Aa - a^\top Bb_0 + u^\top a = -\frac{1}{2}a^\top Aa + (u - Bb_0)^\top a.$$

Thus the conditional information matrix and vector are A and $u - Bb_0$. No block inverse is required to obtain these parameters. The conditional mean still requires solving $A\mu_{a|b_0} = u - Bb_0$.

7.5.2 Marginalization by completing the square

For marginalization, retain every term involving b:

$$p(a, b) \propto \exp\left[-\frac{1}{2}a^\top Aa + u^\top a - \frac{1}{2}b^\top Cb + b^\top(v - B^\top a)\right].$$

Assume C is positive definite. Define $d(a) = v - B^\top a$. The terms in b become

$$-\frac{1}{2}b^\top Cb + b^\top d = -\frac{1}{2}(b - C^{-1}d)^\top C(b - C^{-1}d) + \frac{1}{2}d^\top C^{-1}d.$$

The first term is a centered Gaussian quadratic in b. Its integral is a constant independent of a: shifting its center does not change the integral. The second term remains in the exponent of the marginal. Expand it:

$$\frac{1}{2}(v - B^\top a)^\top C^{-1}(v - B^\top a) = \frac{1}{2}v^\top C^{-1}v - a^\top BC^{-1}v + \frac{1}{2}a^\top BC^{-1}B^\top a.$$

Discarding the constant and collecting the terms in a gives

$$p(a) \propto \exp\left[-\frac{1}{2}a^\top(A - BC^{-1}B^\top)a + (u - BC^{-1}v)^\top a\right].$$

Therefore

$$\boxed{\Lambda_{\text{marg}} = A - BC^{-1}B^\top, \qquad \eta_{\text{marg}} = u - BC^{-1}v.}$$

The matrix $A - BC^{-1}B^\top$ is the SCHUR COMPLEMENT. It accounts for uncertainty in the removed variables. Merely deleting their rows and columns would keep A, which corresponds to conditioning at the level of precision rather than marginalizing.

7.5.3 Calculating a fill edge

Return to the three-variable matrix for (a_1, a_2, b). To retain $a = (a_1, a_2)$ and remove b, take

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \qquad B = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \qquad C = [2].$$

Since $C^{-1} = 1/2$,

$$BC^{-1}B^\top = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \Lambda_{\text{marg}} = \begin{bmatrix} 3/2 & -1/2 \\ -1/2 & 3/2 \end{bmatrix}.$$

The new off-diagonal entry $-1/2$ is the FILL-IN. It replaces the indirect dependence through the removed variable by a direct dependence among retained variables. Inverting gives the same retained covariance as before, $\begin{bmatrix} 3/4 & 1/4 \\ 1/4 & 3/4 \end{bmatrix}$, as marginalization must.

By comparison, conditioning on $b = 0$ gives precision A and covariance $\frac{1}{2}I$. Knowing b exactly removes shared uncertainty and makes a_1 and a_2 independent. Forgetting its coordinate while remaining uncertain about it does not.

7.6 Deriving time prediction in the information filter

An information-form measurement update is additive. Prediction is harder because online filtering removes the previous pose. It is useful to separate that operation into two stages.

1. *Augment:* introduce the next pose and multiply by the motion likelihood, retaining both poses temporarily.
2. *Roll up:* marginalize the previous pose, leaving the new pose and map.

7.6.1 Stage 1: add the next pose

Let the old pose be x and the next pose be y. Linearize the motion model as $y \approx Fx + c + w$, with $w \sim \mathcal{N}(0, R)$ and $c = f(\hat{x}, u) - F\hat{x}$. Partition the old information parameters over (x, m) as

$$\Lambda = \begin{bmatrix} \Lambda_{xx} & \Lambda_{xm} \\ \Lambda_{mx} & \Lambda_{mm} \end{bmatrix}, \qquad \eta = \begin{bmatrix} \eta_x \\ \eta_m \end{bmatrix}.$$

The motion factor has residual $y - Fx - c$. Equivalently, $c = [-F \;\; I \;\; 0](x, y, m)^\top + w'$ with the same noise covariance. Applying the additive linear-factor rule produces, in order (x, y, m),

$$\widetilde{\Lambda} = \begin{bmatrix} \Lambda_{xx} + F^\top R^{-1} F & -F^\top R^{-1} & \Lambda_{xm} \\ -R^{-1} F & R^{-1} & 0 \\ \Lambda_{mx} & 0 & \Lambda_{mm} \end{bmatrix}, \qquad \widetilde{\eta} = \begin{bmatrix} \eta_x - F^\top R^{-1} c \\ R^{-1} c \\ \eta_m \end{bmatrix}.$$

The zeros between y and the map have a meaning: given x, the new motion does not directly depend on landmarks. At this stage only the old pose links the new pose to the map.

7.6.2 Stage 2: remove the old pose

Regroup the augmented variables as retained $a = (y, m)$ and eliminated $b = x$. Apply the Schur-complement formulas just derived. The matrix block for the removed pose has fixed pose dimension, but it can link to many landmarks. The correction $BC^{-1}B^\top$ then touches pairs of those neighbors, creating landmark-landmark links as well as new-pose-to-landmark links.

If there are q active landmarks of fixed dimension, the correction may affect q^2 landmark-block pairs. Thus roll-up can cost quadratically in the active map size and can make that map densely connected. This is the computational cost hidden by the simple instruction to marginalize the old pose.

7.6.3 A scalar prediction worked through

Suppose the old belief over (x, m) has

$$\Lambda = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}, \qquad \eta = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Its mean is $(1, 2)^\top$ and covariance is $\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$. The robot moves according to $y = x + 1 + w$, with motion variance 1. Hence $F = 1$, $c = 1$, and $R = 1$. Augmentation gives

$$\widetilde{\Lambda} = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}, \qquad \widetilde{\eta} = \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}.$$

Retain (y, m) and eliminate x. The retained block is $A = I$, the linking block is $B = (-1, -1)^\top$, the removed block is $C = 3$, and the information-vector blocks are $u = (1, 1)^\top$, $v = -1$. Therefore

$$\Lambda' = \begin{bmatrix} 2/3 & -1/3 \\ -1/3 & 2/3 \end{bmatrix}, \qquad \eta' = \begin{bmatrix} 2/3 \\ 2/3 \end{bmatrix}.$$

Solving gives mean $(2, 2)^\top$ and covariance $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. The robot mean advances by 1, its variance increases by 1, and the map remains unchanged. This is exactly the covariance-form prediction, obtained by augmentation and roll-up.

7.7 Sparsity, approximation, and uncertainty

Repeated observations add active landmarks. Repeated roll-up connects their neighbors. Some information entries may become small, but a small nonzero entry is still a dependency. Exact information-form filtering is therefore not automatically sparse simply because the robot sees only nearby features.

One strategy bounds the number of landmarks directly connected to the robot. It partitions the map into landmarks that remain active, landmarks to be made passive, and those already passive. Making a landmark passive means enforcing conditional independence from the robot given the retained active variables. This changes a relationship in the posterior; it is not just a storage operation.

Sparse extended information filters (SEIFs) approximate the posterior to limit these dependencies. Sparsification must account for how changing one relationship affects the others. Removing selected off-diagonal entries alone is not a general guarantee of a sensible covariance: it can alter positive definiteness, uncertainty in particular directions, and the mean unless the information vector is treated consistently. The approximation studied here can lead to overconfident pose and feature estimates.

Exactly sparse extended information filtering (ESEIF) instead controls the dependencies created in the represented posterior. The term "exactly sparse" describes exact zeros in that representation; it does not mean that nonlinear SLAM has become an exact Gaussian inference problem or that all information is retained. The central aim is to obtain a sparse, statistically consistent estimate rather than reporting unjustifiably small uncertainty.

An accuracy plot alone cannot establish consistency. If an estimator reports covariance P for an error vector e, the normalized estimation error squared is $e^\top P^{-1} e$. A large value can result from a large error or from a covariance that is too small. Under a correctly calibrated Gaussian model with a d-dimensional error, this statistic has a chi-square distribution with d degrees of freedom. That is the distribution of the sum of squares of d independent standard normal variables: scaling the error by its covariance reduces the calculation to this form. Repeatedly exceeding an appropriate statistical bound is evidence that reported uncertainty is too optimistic.

7.8 Keeping old poses: full SLAM as an optimization problem

Marginalization creates fill-in. An alternative is to keep old poses and their local factors. The state becomes larger, but each measurement still touches only a few variables. The distinction is between the number of unknowns and the number of direct relationships; fewer variables do not always imply a cheaper problem.

Let the unknown trajectory be $x_0, \dots, x_T$ and let the map be m. Suppose there are K feature observations. Observation z_k was made at time t_k and corresponds to landmark m_{j_k}. The index k counts observations, while t_k identifies the associated pose. Several observations may share the same time.

Assume an initial-pose prior, a static-map prior $p(m)$, independent motion noises, and independent measurement noises. With controls treated as given inputs and associations treated as known, the posterior factors as

$$p(x_{0:T}, m \mid u_{1:T}, z_{1:K}) \propto p(x_0)p(m) \prod_{i=1}^{T} p(x_i \mid x_{i-1}, u_i) \prod_{k=1}^{K} p(z_k \mid x_{t_k}, m_{j_k}).$$

We now use a locally flat map prior, so $p(m)$ contributes no additional penalty in the region under consideration. If informative map priors are available, their factors must be included instead. The pose prior fixes the coordinate frame. Without an anchor, translating or rotating the whole configuration may preserve all relative measurements, making the system non-unique.

7.8.1 From Gaussian factors to squared errors

Specify the noise models:

$$\begin{aligned} x_0 &\sim \mathcal{N}(\hat{x}_0, P_0), \\ x_i &= f_i(x_{i-1}, u_i) + w_i, & w_i &\sim \mathcal{N}(0, R_i), \\ z_k &= h_k(x_{t_k}, m_{j_k}) + v_k, & v_k &\sim \mathcal{N}(0, Q_k). \end{aligned}$$

Here R_i remains motion covariance and Q_k remains observation covariance, matching the previous lecture. Define the residuals, the discrepancies between a proposed configuration and the data:

$$e_0 = x_0 - \hat{x}_0, \qquad e_i^{\text{mot}} = x_i - f_i(x_{i-1}, u_i), \qquad e_k^{\text{obs}} = z_k - h_k(x_{t_k}, m_{j_k}).$$

For a Gaussian factor with residual e and fixed covariance C,

$$-\log p = \frac{1}{2} e^\top C^{-1} e + \frac{1}{2} \log |C| + \text{constant}.$$

The determinant term is constant with respect to the state when the covariance is fixed. The logarithm of the product of factors is the sum of their logarithms. Maximizing the posterior is

therefore equivalent to minimizing

$$\boxed{J = e_0^\top P_0^{-1} e_0 + \sum_{i=1}^{T} (e_i^{\text{mot}})^\top R_i^{-1} e_i^{\text{mot}} + \sum_{k=1}^{K} (e_k^{\text{obs}})^\top Q_k^{-1} e_k^{\text{obs}}.}$$

The common factor $1/2$ is omitted because it does not change the minimizer. This is a MAXIMUM A POSTERIORI (MAP) estimate. It finds one most probable configuration; it does not by itself return the entire posterior distribution.

The notation $\|e\|_W^2 = e^\top W e$ is called a weighted squared norm. In one dimension, a variance σ^2 contributes e^2/σ^2. An error of two units counts more for a precise sensor than for a noisy one. In multiple dimensions, the covariance also accounts for different scales and correlated noise directions.

7.9 Turning nonlinear residuals into a linear least-squares step

Stack every pose and landmark coordinate into a parameter vector θ. Start from a current guess θ_0 and look for a correction δ, so the next guess will be $\theta_0 + \delta$. For one residual $e(\theta)$,

$$e(\theta_0 + \delta) \approx e(\theta_0) + J_e \delta,$$

where J_e is the residual Jacobian at θ_0. This J_e is a matrix; it is unrelated to the scalar objective J above.

To remove the covariance weight, choose a matrix L with $L^\top L = C^{-1}$. For example, such a factor follows from Cholesky factorization. Then $e^\top C^{-1} e = \|Le\|^2$. Applying this to each residual and stacking the results gives

$$\min_\delta \|A\delta - b\|^2, \qquad A = \begin{bmatrix} L_1 J_{e_1} \\ L_2 J_{e_2} \\ \vdots \end{bmatrix}, \qquad b = -\begin{bmatrix} L_1 e_1(\theta_0) \\ L_2 e_2(\theta_0) \\ \vdots \end{bmatrix}.$$

Here the subscripts enumerate all residual blocks, including the prior. Each row block of A has nonzero columns only for the variables used by its factor. An odometry factor touches two consecutive poses. A landmark factor touches one pose and one landmark. This is the source of sparsity.

After solving for δ, update the guess and recompute residuals and Jacobians. For linear models one solve suffices. For nonlinear models this is a local iterative method, and its success depends on the starting guess and the quality of the linear approximation. A small correction is meaningful only if the resulting configuration actually improves the nonlinear fit.

7.9.1 Deriving the normal equations

Expand the squared norm:

$$\|A\delta - b\|^2 = \delta^\top A^\top A\delta - 2\delta^\top A^\top b + b^\top b.$$

Differentiating with respect to δ and setting the gradient to zero yields

$$A^\top A\delta = A^\top b.$$

These are the normal equations. If A has full column rank, meaning its columns are linearly independent, the minimizer is unique. A missing global anchor or an unobserved degree of freedom can cause rank deficiency. Writing an inverse does not resolve such missing information.

7.9.2 Why use QR factorization?

For a tall, full-column-rank matrix, write $A = Q\begin{bmatrix}\mathcal{R}\\0\end{bmatrix}$, where Q is orthogonal and $\mathcal{R}$ is upper triangular. Orthogonal transformations preserve Euclidean length, so

$$\|A\delta - b\|^2 = \left\|\begin{bmatrix}\mathcal{R}\\0\end{bmatrix}\delta - Q^\top b\right\|^2.$$

Partition $Q^\top b = (c_1, c_2)^\top$ to match the row blocks. The objective becomes $\|\mathcal{R}\delta - c_1\|^2 + \|c_2\|^2$. The second term does not depend on δ. Solve the triangular system $\mathcal{R}\delta = c_1$ by back substitution.

QR avoids explicitly forming $A^\top A$, which squares the condition number for a full-rank system. The condition number measures sensitivity to small perturbations; a large value signals that small data or rounding changes can cause large solution changes. This can matter when different constraints produce poorly scaled or nearly dependent directions. The symbols Q and $\mathcal{R}$ here denote factorization matrices, not sensor or motion covariances. Sparse ordering still matters: eliminating a variable can create fill even when the original factors are local.

7.10 Worked loop closure: how a discrepancy is distributed

Consider a scalar robot trajectory with the starting position fixed at $x_0 = 0$. Two odometry measurements each report a displacement of 1. A loop-closure constraint reports that the endpoint displacement from the start is $9/5 = 1.8$. The odometry path alone would end at 2, so the three constraints cannot all be satisfied exactly. The following invented example makes the optimization explicit.

7.10.1 Equal measurement variances

With all three noise variances equal to 1, minimize

$$J(x_1, x_2) = (x_1 - 1)^2 + (x_2 - x_1 - 1)^2 + (x_2 - 9/5)^2.$$

This already is a linear least-squares problem. With $\theta = (x_1, x_2)^\top$,

$$A = \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 1 \\ 9/5 \end{bmatrix}, \qquad A\theta - b = \begin{bmatrix} x_1 - 1 \\ x_2 - x_1 - 1 \\ x_2 - 9/5 \end{bmatrix}.$$

Multiplication gives

$$A^\top A = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad A^\top b = \begin{bmatrix} 0 \\ 14/5 \end{bmatrix}.$$

Thus $2x_1 - x_2 = 0$ and $-x_1 + 2x_2 = 14/5$. The first gives $x_2 = 2x_1$. Substitution into the second gives $3x_1 = 14/5$, so

$$x_1 = \frac{14}{15}, \qquad x_2 = \frac{28}{15}.$$

The residuals are $-1/15, -1/15, +1/15$. Each odometry displacement is shortened slightly, while the endpoint remains slightly beyond the loop-closure reading. The optimization balances all three constraints according to their equal precision.

7.10.2 A more precise loop-closure measurement

Now give the loop closure variance $1/4$, keeping both odometry variances equal to 1. Its precision is 4, so its squared residual receives weight 4:

$$J = (x_1 - 1)^2 + (x_2 - x_1 - 1)^2 + 4(x_2 - 9/5)^2.$$

The first normal equation remains $2x_1 - x_2 = 0$. The second becomes $-x_1 + 5x_2 = 41/5$. Therefore $x_1 = 41/45$ and $x_2 = 82/45$. The endpoint is now closer to 1.8. The residuals are $-4/45, -4/45, +1/45$: the more precise measurement is fitted more closely, while the less precise motion constraints absorb more of the discrepancy.

This is the role of uncertainty in graph optimization. A loop closure is not automatically enforced exactly, and old poses are not frozen. The solution adjusts all connected unknowns to fit the available constraints with their stated precision.

Source. Matthew R. Walter, TTIC 31170, Lecture 7: Simultaneous Localization and Mapping (continued), October 19, 2026.